International Journal of Information Security (2026) 25:107
https://doi.org/10.1007/s10207-026-01284-4

REGULAR CONTRIBUTION

Structuring trust: a quantitative and traceable framework for hardware security assurance

Shao-Fang Wen[1,2] · **Arvind Sharma**[1]

Received: 12 November 2025 / Accepted: 5 June 2026 / Published online: 12 June 2026
© The Author(s) 2026

Abstract

The security assurance of hardware systems is increasingly critical in connected and safety-sensitive infrastructures, where compromised components may endanger human safety or disrupt essential services. However, existing assurance practices remain largely qualitative and fragmented across overlapping standards, often leading to duplicated evaluation efforts and inconsistent interpretations of system trustworthiness. This paper introduces a structured, multi-level framework for hardware security assurance that links security requirements to verification evidence through six traceable layers. The framework establishes a traceable and structured evaluation process designed to enhance reproducibility and introduces a quantification-ready, structured aggregation methodology for analyzing subsystem assurance evidence. By integrating system decomposition, standards-derived requirements, and evidence-based verification, the approach supports transparent traceability across complex, multi-component hardware systems. The framework is demonstrated through a case study of a wireless fingertip oximeter used in healthcare infrastructure. The results illustrate how traceable assurance artifacts and structured aggregation can support measurable subsystem evaluation and provide diagnostic insight into system-level resilience.

Keywords Hardware security assurance · Quantitative assurance · Security metrics · Security evaluation

1 Introduction

The security assurance of hardware-based systems has become a critical concern across safety–critical and highly connected domains. Medical devices, industrial controllers, and embedded platforms increasingly underpin infrastructures whose disruption can affect public safety, healthcare delivery, or industrial continuity. These systems operate in environments exposed to persistent cyber threats, while regulators require robust evidence that the underlying components can be trusted to perform securely and reliably [1–3]. Failures in hardware assurance may therefore produce cascading consequences, ranging from compromised patient safety in clinical settings to large-scale disruptions across interconnected infrastructure systems. In such contexts, assurance is not merely a regulatory requirement but a systematic process of establishing justified confidence that systems have been designed, implemented, and validated against recognized threats [4].

Despite its importance, existing assurance practice often struggles to provide this confidence in a consistent and transparent manner. Many evaluations still rely heavily on qualitative reasoning or assessor-dependent judgment, introducing subjectivity and limiting reproducibility [5, 6]. Security standards offer authoritative requirements, yet they are typically developed independently across domains and lifecycle phases. When applied in combination, these standards may overlap, leave gaps, or employ inconsistent abstraction levels [7, 8]. As a result, evaluators frequently face duplicated effort and uncertainty about whether assurance outcomes represent a coherent and defensible view of system trustworthiness [9].

These challenges are further amplified by the complexity of modern hardware ecosystems. Hardware components are commonly designed, fabricated, and integrated across globally distributed supply chains involving multiple vendors and manufacturing stages. This distribution complicates the definition of clear evaluation boundaries and

✉ Shao-Fang Wen
shao-fang.wen@ntnu.no

[1] Department of Information Security and Communication Technology, Norwegian University of Science and Technology, 2815 Gjøvik, Norway

[2] Department of Business and IT, University of South-Eastern Norway, 3800 Bø, Norway

raises questions about which elements should fall within the scope of assurance [10, 11]. Issues such as component provenance, fabrication processes, and integrated-circuit development introduce additional uncertainty into assurance reasoning. Moreover, many existing evaluation frameworks remain predominantly qualitative and provide limited analytical support for comparing subsystem assurance results or consolidating them into a traceable system-level perspective [12].

Recent academic research has attempted to address aspects of these limitations. Quantitative security metrics have been proposed to measure aspects of system resilience [12, 13], while hardware-focused evaluation frameworks have explored structured approaches to subsystem analysis [14]. Other studies have developed techniques for measuring hardware leakage and side-channel resistance [15]. Although these contributions provide valuable insights, they are often scoped to specific device classes, threat models, or evaluation contexts. Consequently, their applicability to heterogeneous infrastructure systems remains limited, and a unified framework capable of supporting structured assurance reasoning across complex hardware architectures is still lacking.

These observations reveal four persistent challenges in achieving traceable and reproducible hardware security assurance. First, abstract security objectives must be systematically refined into measurable verification evidence in order to avoid vague or subjective assurance claims [16–18]. Second, the structural complexity and heterogeneity of hardware systems require clearly defined evaluation boundaries that remain meaningful at both component and system-level [19]. Third, requirements derived from overlapping standards must be integrated into a coherent and traceable structure while also addressing assurance gaps in areas such as supply chain authenticity, lifecycle obsolescence, and emerging microarchitectural threats [20]. Finally, assurance methodologies must move beyond purely qualitative reasoning toward analytic approaches capable of supporting measurement, subsystem comparison, and structured aggregation into a system-level view of infrastructure resilience [21, 22].

Together, these issues point to a fundamental methodological gap in current hardware security assurance practice. Although existing standards, evaluation frameworks, and academic models each contribute important elements, they rarely provide a unified mechanism that connects system decomposition, requirement derivation, verification evidence, and analytical evaluation within a single traceable structure. As a result, assurance activities are often performed as loosely connected tasks rather than as components of a coherent reasoning framework. Addressing this gap requires an architectural approach that systematically links system scope, security objectives, verification artifacts, and evaluation outcomes.

To address this need, this paper proposes a structured framework for hardware security assurance that links system scope, assurance artifacts, and verification evidence within a traceable evaluation architecture. The framework organizes assurance reasoning into six hierarchical layers that progressively refine system boundaries into verifiable security evidence. Beginning with the definition of the system of interest, the framework decomposes the architecture into evaluation units, derives security objectives and criteria, formulates concrete requirements, and associates these requirements with explicit verification conditions. Each layer therefore represents a controlled refinement step in which abstract protection goals are translated into increasingly concrete and verifiable assurance artifacts. Through this hierarchical mapping, system-level objectives are systematically connected to observable verification evidence, enabling transparent reasoning about how subsystem protections contribute to overall system assurance.

Beyond structural traceability, the framework also supports analytical reasoning about assurance outcomes. Verification evidence produced at subsystem-level can be consolidated through a structured aggregation methodology that enables evaluators to reason about the broader security posture of the system. Rather than introducing standalone numerical metrics detached from verification logic, these indicators emerge directly from requirement-to-verification mappings. This approach preserves interpretability while enabling structured comparison across subsystems and providing diagnostic insight into system-level assurance.

The proposed framework differs from prior assurance case methodologies and standards-based evaluation approaches in three key respects. First, it introduces Assurance-Oriented Hardware Decomposition (AHD), a deterministic method for partitioning hardware systems into evaluation units aligned with assurance-relevant architectural semantics rather than implementation hierarchy. Second, it establishes an explicit six-layer traceability structure that connects system scope, evaluation units, objectives, criteria, requirements, and verification conditions within a single integrated architecture. Third, the framework incorporates a quantification-ready aggregation methodology that enables subsystem-level verification evidence to be consolidated into structured system-level indicators while preserving traceability to the underlying assurance artifacts.

The contributions of this paper are threefold. First, it conceptualizes a multi-level hardware security assurance framework that operationalizes structured system decomposition, traceable assurance derivation, and evidence-based verification while integrating heterogeneous standards and addressing emerging assurance gaps. Second, the framework is demonstrated through a case study involving a wireless fingertip oximeter used in healthcare infrastructure, illustrating how assurance artifacts can be systematically derived

across multiple levels and linked to verification evidence. Third, the work introduces a structured aggregation methodology that enables comparison across subsystems and supports the consolidation of verification outcomes into a system-level view of infrastructure resilience.

The remainder of this paper is organized as follows. Section 2 reviews related work and establishes the theoretical foundations of the proposed approach. Section 3 introduces the design pillars that guide the framework design. Section 4 presents the multi-level framework architecture and its hierarchical traceability model. Section 5 describes the evaluation methodology used in the case study. Section 6 presents the case study results. Section 7 extends the framework with a structured aggregation methodology for analyzing subsystem assurance evidence. Section 8 discusses implications, compares the framework with existing approaches, and outlines limitations and future work. Section 9 concludes the paper.

2 Theoretical background

This section establishes the theoretical and methodological foundations that inform the design of the proposed framework. It synthesizes concepts from security assurance theory, international standards, and prior academic research in order to identify structural gaps in current approaches to hardware security assurance. By examining both standards-driven practice and research-based methodologies, the section highlights limitations in existing assurance models, including fragmentation across assurance dimensions, insufficient traceability between requirements and verification evidence, and the absence of integrated structural frameworks capable of supporting system-level reasoning. These observations motivate the design principles and architectural structure of the framework introduced in Sect. 4.

2.1 Security assurance in hardware systems: definition and challenges

Security assurance refers to a disciplined process of evaluating and substantiating the trustworthiness of a system, with the objective of demonstrating that defined security requirements are satisfied and that the system can resist relevant threats across its lifecycle [23, 24]. More specifically, it reflects the degree of confidence that these requirements are fulfilled in a substantive manner, while any residual vulnerabilities are either explicitly accepted within an agreed risk tolerance or mitigated through verifiable countermeasures [25, 26]. In practice, assurance means more than demonstrating compliance; it requires showing that a system is effectively protected against threats, weaknesses, and attacks by maintaining appropriate safeguards [21]. At its core, assurance provides justified confidence, supported by structured evidence and defensible reasoning that connect high-level security objectives to observable verification outcomes [20].

When applied to hardware, assurance extends beyond software correctness to encompass the physical and architectural properties of devices. It requires evaluation across multiple layers, from printed circuit boards and chips to firmware, software, and operational environments [27]. Key features include tamper resistance, secure boot, cryptographic acceleration, vulnerability management, and incident response, all of which contribute to resilience against advanced attacks. Hardware assurance must also account for physical exposure, subsystem heterogeneity, extended lifecycles, and supply chain risks. Recent studies highlight these imperatives: Mishra and Sahay [28] survey emerging threats such as fault injection, voltage glitching, and speculative execution flaws, while NIST IR 8517 [29] catalogs failure scenarios tied to integration and lifecycle weaknesses. Taken together, these findings underscore that hardware assurance must integrate both logical and physical dimensions to provide enduring trustworthiness.

While this broader scope defines what hardware assurance entails, its practical realization is constrained by four recurring challenges. First, hardware is uniquely exposed to physical attacks. Adversaries can manipulate voltage, clock signals, or electromagnetic fields to induce faults, and exploit side channels such as power consumption and timing to extract secrets [30, 31]. Addressing these threats requires assurance activities that extend beyond code review into fault injection testing, environmental analysis, and physical validation.

Second, hardware platforms are increasingly heterogeneous, combining processors, secure elements, networking interfaces, and firmware controllers within a single device. Preserving subsystem context while reasoning at the system-level is critical, since vulnerabilities often emerge at integration boundaries. Studies emphasize that secure composition demands explicit support for cross-component threats [32, 33], and methods such as layered fault-tree analysis demonstrate how assurance can span hardware and software layer [34].

Third, hardware devices often operate under extended lifecycles. Medical, automotive, and industrial platforms may remain in service for decades, during which threats, cryptographic standards, and regulations evolve. This creates assurance challenges around secure updates, obsolescence, and long-term trust. Studies reveal widespread vulnerabilities in end-of-life devices [34], while frameworks for obsolescence management [35] and regulatory guidance on legacy medical devices [36] stress the need for continuous assurance across the total product lifecycle.

Finally, hardware assurance involves specialized verification methods beyond traditional testing. Techniques such as side-channel analysis, fault injection, and formal verification (e.g., SecVerilog) are required to validate critical properties under adversarial conditions [37]. Yet aligning such diverse forms of evidence into a coherent assurance argument remains unresolved.

These difficulties collectively demonstrate the need for frameworks that can facilitate integration into system-level reasoning, capture subsystem context, and account for physical, lifespan, and verification restrictions in hardware assurance. The ensuing sections examine how these problems are addressed by current standards and scholarly works, as well as the reasons why further structuring is required.

2.2 Role of standards

Standards form the authoritative foundation for hardware security assurance, setting expectations for the protections and controls required in regulated domains. For example, FIPS 140-3 specifies cryptographic module protections such as tamper resistance, key management, and lifecycle controls [38]. IEC 62304 governs the software lifecycle of medical devices but inevitably extends into the hardware-software interface, where embedded controllers and processors are safety–critical [39]. In the medical domain, ISO 14971 prescribes risk management practices [40], while UL 2900-2-1 defines cybersecurity requirements for network-connectable healthcare products [41]. At a broader organizational level, ISO/IEC 27001 establishes information security management principles that also influence hardware assurance [42]. Together, these standards define the baseline requirements against which security claims are evaluated.

Although authoritative, security standards have evolved largely in isolation, leading to fragmentation, overlaps, and unaddressed scope. For example, mechanisms such as secure boot and cryptographic validation appear in multiple standards but are described with inconsistent terminology and varying rigor, complicating reconciliation in practice. At the same time, important risks are under-specified: supply chain authenticity, lifecycle obsolescence, and resilience to microarchitectural attacks are unevenly treated or absent altogether. Therefore, complete assurance cannot be guaranteed by merely adhering to one or even multiple standards.

The history of hardware vulnerabilities reinforces this point. Side-channel exploits such as Spectre and Meltdown revealed weaknesses in speculative execution that were not previously considered in standards [43]. Likewise, CWE-1247 documents fault injection vulnerabilities from voltage and clock manipulation, showing that certain hardware-level threats extend beyond the scope of existing requirements [44]. Academic studies echo these limitations, noting that standards often function as baselines that are necessarily reactive and incomplete.

In this paper, standards are treated as critical inputs to assurance rather than as complete solutions. They provide the authoritative requirements that must be respected, but the diversity and fragmentation of these documents mean they must be consolidated, reconciled, and aligned to form a coherent assurance argument. This motivates the need for a structured approach, introduced later in this work, that organizes heterogeneous standards into a unified requirements set while preserving their provenance and supporting evidence-based evaluation.

2.3 Related academic work

Research into security assurance has yielded a variety of frameworks, methodologies, and surveys, but their coverage of hardware remains fragmented. Several reviews emphasize this gap. Yaacoub et al. [45] highlight that cyber-physical systems security research often addresses narrow aspects such as encryption or network security, without unifying them into a broader assurance methodology. Similarly, Shukla et al. [5] identify that existing assurance approaches for ICT and CPS are either highly abstract, offering conceptual models with limited operational guidance, or highly specific, focusing on isolated domains such as microcontrollers or IoT devices. Both reviews converge on the point that a unified, transferable framework is lacking.

Table 1 summarizes representative academic contributions across several key dimensions of hardware security assurance. These works address important aspects of assurance, including foundational protection mechanisms, subsystem heterogeneity, lifecycle trust management, and verification methodologies. Collectively, they demonstrate substantial progress in identifying hardware security risks and developing techniques for mitigating them.

Nevertheless, these contributions largely address individual assurance dimensions in isolation. Many studies focus on specific protection mechanisms, such as side-channel resistance or secure boot enforcement, while others concentrate on specialized evaluation methodologies, including formal verification or adversarial testing. As a result, the literature provides limited support for integrating heterogeneous assurance artifacts into a coherent system-level reasoning structure. This fragmentation reveals a fundamental methodological gap: the absence of a structured assurance framework capable of simultaneously supporting system decomposition, traceable derivation of objectives and requirements, and evidence-based verification across heterogeneous hardware subsystems.

The above limitations identified across both standards practice and academic research suggest that effective hardware security assurance requires several structural

Table 1 Summary of representative academic work in hardware security assurance

Assurance dimension	Representative mechanisms/Focus	Representative academic work	Contribution
Foundational Security Mechanisms	Tamper resistance, secure boot, cryptographic acceleration, vulnerability management, risk assessment, incident response	[46–51]	Hardware Trojan detection; Taxonomies of microarchitectural vulnerabilities; framing assurance against physical attacks
Subsystem Heterogeneity	Integration of processors, secure elements, networking interfaces, firmware controllers	[32, 52, 53]	Compositional assurance linking hardware/software layers; preserving subsystem context
Lifecycle & Long-Term Trust	Update mechanisms, obsolescence management, maintaining trust across medical/industrial/automotive device lifetimes	[35, 36, 54]	Scenario-based obsolescence management; regulatory guidance on total product lifecycle security
Verification & Evidence	Formal verification of information flows; side-channel and fault-injection testing	[15, 37, 55, 56]	Formal methods and systematic adversarial testing to produce reproducible evidence
Toward Quantitative & Automated	Quantitative scoring, AI-driven requirement elicitation and traceability	[12–14, 27, 57–60]	Hardware security scoring framework; AI/ML support for requirement extraction and assurance automation

capabilities. First, assurance must support system-level decomposition, enabling evaluators to reason about heterogeneous subsystems while preserving architectural context. Second, assurance artifacts must maintain explicit traceability relationships linking objectives, requirements, and verification evidence, a principle widely emphasized in assurance case methodologies and safety–critical system evaluation. Third, assurance reasoning must support aggregation across subsystems, allowing evaluators to assess the overall security posture of the system rather than isolated technical components.

These observations motivate the design pillars that guide the framework architecture presented later in this paper: boundary-driven system decomposition, hierarchical traceability of assurance artifacts, and evidence-based verification capable of supporting reproducible assurance evaluation.

2.4 Boundary-driven system of interest model

A central difficulty highlighted in both standards and academic work is that assurance often lacks a clear and defensible definition of system scope. Without explicit boundaries, evaluations risk either including elements irrelevant to the stated objectives or excluding critical dependencies whose compromise would undermine trustworthiness. To address this, the proposed framework is grounded in the Boundary-Driven System-of-Interest (SoI) Model, originally introduced by Wen and Katt in the context of software security assurance evaluations [25, 61]. The SoI refers to the specific device, component, or system identified by stakeholders as the focus of assurance activities, with its boundary preventing ambiguity and ensuring that evaluation efforts remain directed and relevant.

The SoI Model used in this research is illustrated in Fig. 1. In its extended form, the model is structured around a Component-Environment-Process (C-E-P) triad. The *System Component* comprises the core hardware under direct evaluation (e.g., chips, firmware, controllers). The *System Environment* includes integrated or peripheral elements whose trustworthiness is essential but not the primary focus of evaluation (e.g., networks, power supply). The extension introduces a *System Process* layer, capturing workflows such as provisioning, manufacturing, update deployment, and incident response. These processes directly affect system trustworthiness throughout the lifecycle, as a compromised provisioning pipeline or insecure update service can bypass otherwise robust technical safeguards.

The inclusion of the process dimension addresses an important limitation in many existing assurance frameworks, which typically emphasize technical artifacts while leaving operational workflows implicit. For instance, NIST SP 800-193

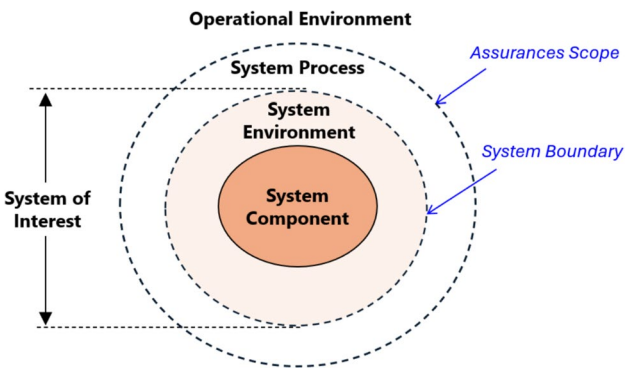


Fig. 1 The boundary-driven SoI model

provides comprehensive guidance on firmware integrity and recovery but does not explicitly address how provisioning or update workflows are secured over time. Similarly, the Trusted Computing Group (TCG) device trust model [62] focuses on hardware roots of trust and cryptographic anchors but does not integrate supply chain or patch management processes. PSA Certified defines baseline goals for IoT device security, yet lifecycle processes are largely abstracted away and assumed to be handled externally.

By explicitly incorporating *System Process* such as provisioning, secure manufacturing, update distribution, patching, and auditing, the SoI model captures assurance dependencies that extend beyond the initial design phase. These operational processes are frequently exploited by adversaries. For example, compromised provisioning pipelines may enable the creation of rogue device identities, while weak update signing mechanisms may permit rollback attacks. Evaluating such processes may involve a combination of device-level testing, such as verifying that only authenticated updates are accepted, and vendor-provided evidence including process documentation, audit logs, or certification artifacts. Including these processes within the assurance boundary ensures that the framework reflects not only static hardware protection but also the dynamic operational mechanisms that sustain trustworthiness throughout the device lifecycle.

In this study, the practical instantiation of the SoI model focuses primarily on device-level processes that are directly coupled with technical assurance evaluation, including provisioning, update deployment, and incident response. Upstream supply chain activities, such as semiconductor fabrication and component sourcing, are acknowledged conceptually as part of the broader assurance ecosystem but are not subjected to direct verification within the case evaluation presented in this work. This scope definition maintains methodological clarity while preserving the conceptual completeness of the SoI model.

By making system boundaries explicit through the C-E-P triad, the SoI model enables assurance evaluations that are focused, auditable, and reproducible. It allows evaluators to distinguish clearly between elements that fall inside the assurance boundary and are therefore subject to verification, and those that remain outside the boundary and must instead be assumed or constrained. By integrating environmental dependencies and operational processes alongside component-level evaluation, the model supports a more holistic understanding of system assurance that extends beyond traditional component-centric approaches.

3 Design pillars

The design of a security assurance framework cannot be arbitrary or ad hoc. It must rest on a set of foundational pillars that capture enduring principles of how assurance should be conceived, justified, and practiced. These pillars reflect insights from established theories and prior research but also respond to the unique challenges of assuring hardware systems. They represent the intellectual backbone of our philosophy of assurance, distilled through academic practice and honed into four interrelated commitments: progression, boundary and unit-based scoping, traceable standards integration, and analytic assurance. Figure 2 illustrates these four design pillars and their interrelationships, serving as the conceptual foundation of the proposed framework.

Pillar 1: Progression from Abstract to Concrete

Security assurance must translate high-level intent into observable evidence. Established principles in requirements engineering emphasize systematic refinement of abstract objectives into measurable and testable conditions to ensure completeness and reproducibility [16]. Similarly, the Goal-Question-Metric paradigm formalizes the transformation of strategic goals into operational metrics [17], while assurance case theory requires explicit linkage between claims and supporting evidence to maintain credibility [18].

In hardware security contexts, objectives such as preserving firmware integrity or protecting cryptographic keys cannot remain declarative. They must be decomposed into concrete requirements and validated through defined tests or measurements. By enforcing a disciplined progression from intent to evidence, assurance becomes reproducible across evaluators and resistant to subjective interpretation. This pillar therefore establishes operationalization as the foundation of credible assurance reasoning.

Pillar 2: Boundary- and Unit-Based Scoping

Effective assurance depends on explicit system boundaries and well-defined units of evaluation. System boundary theory emphasizes the need to distinguish clearly between elements inside and outside the scope of assessment to avoid both oversight and unnecessary expansion [19, 25, 61]. In hardware security, assurance challenges vary across architectural layers, making it essential to anchor evaluation to a concrete and structurally bounded unit.

For example, assessing a cryptographic module requires deciding whether the scope covers only the silicon, the surrounding board, or associated lifecycle processes. Each scope choice implies different verification techniques, such as side-channel analysis at the chip level or secure update validation at the firmware level. By defining defensible boundaries and non-overlapping units, this pillar ensures that assurance remains both technically appropriate and methodologically coherent.

Pillar 3: Traceable Standards Integration

Assurance claims must be grounded in authoritative standards to avoid ad hoc justification. Traceability ensures

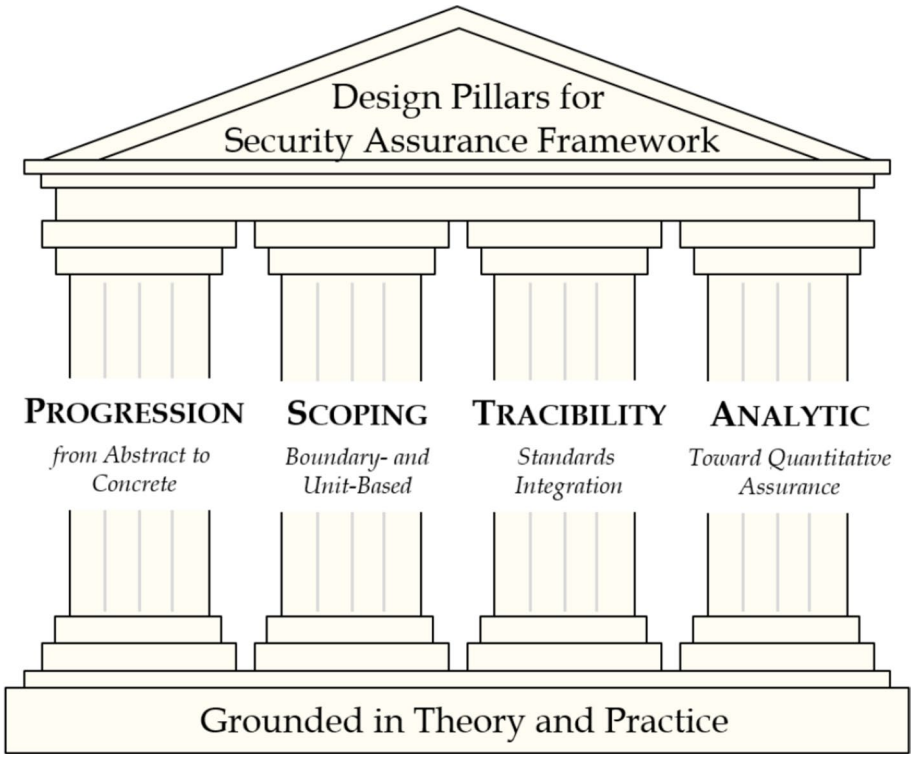


Fig. 2 The four design pillars of the security assurance framework

that every requirement and verification step can be linked to a recognized normative source, providing an auditable chain of reasoning for regulators and evaluators [19, 20].

In hardware contexts, multiple standards often apply simultaneously, such as FIPS 140-3 for cryptographic modules [38], IEC 62304 governs lifecycle integrity for medical device software [39], and UL 2900-2-1 defines cybersecurity protections for network-connectable healthcare systems [41]. Because these standards are developed independently, their combined application can introduce overlap or inconsistency.

Traceable standards integration addresses this challenge by reconciling heterogeneous requirements within a unified structure while preserving their provenance. This structured consolidation reduces redundancy, prevents contradiction, and maintains defensible alignment with normative obligations.

Pillar 4: Assurance Analytics

Assurance reasoning benefits from systematic analysis rather than purely qualitative judgment. Evidence-based evaluation enables comparison, aggregation, and identification of residual gaps [20, 21]. Quantitative approaches extend this capability by transforming verification outputs into analyzable metrics that capture subsystem coverage and structural weaknesses [12, 22].

In hardware security, where heterogeneous components introduce varying risk profiles, analytics enable consistent comparison across evaluation units. Aggregated evidence reveals patterns that may not be visible at the individual control level, supporting structured prioritization and resilience assessment. By converting unit-level verification into comparable indicators, this pillar extends assurance beyond compliance checking toward diagnostic insight.

Integrative perspective

The four pillars define the conceptual foundation of the framework. Progression operationalizes intent into evidence; scoping establishes defensible architectural boundaries; traceability anchors assurance in authoritative standards; and analytics enable structured reasoning across subsystems. Table 2 summarizes how these pillars address the challenges identified in Sect. 2 and situate them within established assurance theory.

4 Framework architecture

The framework architecture operationalizes the design pillars into a structured, hierarchical model for hardware security assurance. As illustrated in Fig. 3, the framework is organized along two dimensions: (i) analytical segments that represent stages of assurance reasoning, and (ii) hierarchical levels (L1-L6) that formalize the refinement from system boundary to verifiable evidence.

Table 2 Design pillars, addressed challenges, and theoretical foundations

Design Pillar	Addressed Problem / Challenge	Theoretical Anchor
Pillar 1: Progression from abstract to concrete	Assurance remains qualitative; objectives expressed vaguely and assessed subjectively, undermining reproducibility	Requirements engineering refinement [16]; Goal-Question-Metric paradigm [17]; Assurance case theory [18]
Pillar 2: Boundary- and unit-based scoping	Hardware systems are heterogeneous; unclear scope leads to overlooked dependencies or wasted effort; assurance varies across levels (chip, firmware, subsystem)	System boundary theory [19]; SoI concepts in assurance practice [25, 61]
Pillar 3: Traceable standards integration	Fragmented, overlapping standards (FIPS, IEC, UL, ISO) create duplication, contradictions, and inconsistency in evaluation	Traceability research [19]; Assurance argumentation [20]
Pillar 4: Assurance analytics	Predominantly qualitative frameworks lack metrics for coverage, aggregation, and residual risk discovery; supply chain complexity exacerbates this gap	Assurance reasoning with evidence [21]; Standards integration and traceability [12, 20, 22]

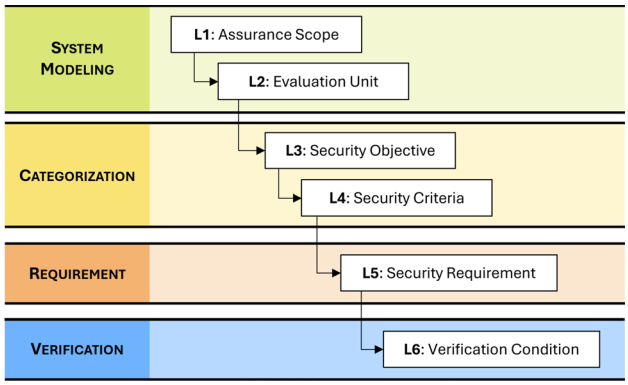


Fig. 3 The multi-level hardware security assurance framework

Assurance reasoning within the framework proceeds strictly in a top-down derivation process, where artifacts at each level are derived from the immediately preceding level and must satisfy explicit structural constraints. This hierarchical derivation guarantees that all assurance artifacts remain anchored to the defined system boundary while preserving traceability across architectural elements, security objectives, requirements, and verification evidence.

Table 3 summarizes the formal role, inputs, outputs, and structural constraints associated with each hierarchical level of the framework. The following subsections describe each hierarchical level in detail, beginning with system modeling and proceeding through objective definition, requirement specification, and verification.

4.1 System modeling

The System Modeling segment establishes the architectural foundation for assurance. It defines the system boundary and partitions it into structurally coherent evaluation units. No security objectives, requirements, or verification constructs are introduced at this stage; the purpose is purely structural formalization.

4.1.1 Level 1: assurance scope

Level 1 defines the formal boundary of evaluation. Let S denote the scoped SoI. The assurance scope is specified using C-E-P model introduced in Sect. 2.3, which captures three essential dimensions: included components, relevant environmental dependencies, and lifecycle processes influencing assurance.

Formally, the output of Level 1 is a bounded system:

$$S = \{C, E, P\}$$

Table 3 Formal definition of framework layers (L1-L6)

Level	Assurance Layer	Formal Role	Input	Output	Structural Constraint
L1	Assurance Scope	Defines the boundary of the System of Interest using the C-E-P model	System description and stakeholder context	Scoped System of Interest S	Boundary must explicitly enumerate components, environment, and processes
L2	Evaluation Unit	Partitions the scoped system into structurally coherent assurance units	Scoped system S	Set of evaluation units $U = \{U_1, U_2, \ldots, U_n\}$	Units must be non-overlapping and collectively cover the scoped system
L3	Security Objective	Defines high-level protection goals for each evaluation unit	Evaluation unit U_i	Set of objectives O_{ij}	Each objective must belong to exactly one evaluation unit
L4	Security Criterion	Refines objectives into measurable assurance domains	Objective O_{ij}	Set of criteria C_{ijk}	Each criterion derives from a single objective
L5	Security Requirement	Specifies concrete and testable security controls	Criterion C_{ijk}	Requirements R_{ijkl}	Each requirement must be traceable to a normative source
L6	Verification Condition	Defines observable evidence used to validate requirements	Requirement R_{ijkl}	Verification conditions V_{ijklm}	Each requirement must have at least one verification condition

where each included element is explicitly enumerated and external dependencies are identified but not evaluated unless brought within scope.

This boundary definition determines the admissibility of requirements, the sufficiency of evidence, and the architectural locations to which assurance artifacts may later attach. Explicit scoping therefore serves as a prerequisite for reproducibility and prevents ambiguity in downstream assurance reasoning.

4.1.2 Level 2: evaluation unit

Level 2 transforms the assurance scope defined at Level 1 into a finite set of evaluation units. An evaluation unit is a structurally bounded and non-overlapping portion of the system that serves as a stable container for subsequent assurance reasoning. At this level, units are defined purely in structural terms: they do not yet carry security objectives, requirements, or verification conditions. Their role is to establish where assurance reasoning may later be applied in a consistent, traceable, and reproducible manner.

Formally, Level 2 transforms the scoped system S into a finite set of structurally bounded evaluation units:

$$U = \{U_1, U_2, \ldots, U_n\}$$

Each evaluation unit functions as a stable structural container for later assurance constructs. Units are defined purely in architectural terms and do not yet carry objectives, requirements, or verification conditions. Their role is to establish where assurance reasoning may subsequently be applied in a consistent and traceable manner.

The set U must satisfy three invariants:

- Structural coherence: internally consistent architectural semantics.
- Non-overlap: $U_i \cap U_j = \emptyset$ for $i \neq j$.
- Collective coverage: $\bigcup_i U_i = S$.

These constraints ensure that the system is neither redundantly partitioned nor partially uncovered.

Assurance-oriented hardware decomposition

The derivation of evaluation units is performed through Assurance-Oriented Hardware Decomposition (AHD). AHD executes a deterministic transformation from the scoped system S to the set U. The transformation is governed by assurance relevant structural semantics rather than functional design hierarchy. This distinction is critical. Engineering decomposition reflects implementation intent, whereas AHD identifies structurally stable regions aligned with trust transitions, interaction surfaces, and evidential separability.

AHD proceeds through fixed rule-governed operations:

- Enumeration of architectural elements and interfaces within scope
- Identification of trust transitions and exposure surfaces
- Partitioning based on evidential separability
- Iterative consolidation until structural stability is achieved

The decomposition terminates when no further split or merge operation can be justified without violating structural

coherence or evidential independence. Table 4 summarizes the formal logic of AHD.

AHD operates exclusively at the structural level. By separating structural modeling from normative reasoning, the framework preserves determinism and prevents premature mixing of architectural and security semantics.

All subsequent artifacts introduced at Levels 3–6 must be attached to exactly one evaluation unit:

$$\forall A_x \in \{O, C, R, V\}, \exists!\, U_i$$

Each artifact has exactly one structural host.

This constraint enforces structural traceability and guarantees that assurance reasoning remains anchored to a single, well defined architectural region.

4.2 Categorization

The Categorization segment embodies the principle of progression (operationalization). By distinguishing between high-level objectives and their systematic refinement into measurable criteria, it ensures that assurance reasoning does not remain abstract but advances toward specificity. This structured descent makes it possible to demonstrate completeness, minimize redundancy, and prepare for traceable mapping to requirements.

4.2.1 Level 3: security objective

For each evaluation unit U_i, the framework defines a finite set of security objectives:

$$O_i = \{O_{i1}, O_{i2}, \dots\}$$

Security objectives articulate the high-level protection goals associated with the evaluation unit. They specify what must be preserved or protected within the architectural boundary of the unit while remaining independent of specific implementation mechanisms. In this sense, objectives capture the intended security posture of the unit and guide the subsequent refinement of assurance artifacts.

Objectives therefore frame the central assurance questions for each unit, including which assets require protection, which threat classes must be mitigated, and which security properties must be maintained. By expressing protection intent at an abstract level, objectives establish the conceptual foundation from which measurable assurance criteria and concrete requirements can later be derived.

For example, within a secure boot unit, a security objective may require that firmware integrity be preserved against unauthorized modification. In a cryptographic subsystem, an objective may specify the preservation of key confidentiality and operational integrity. Such objectives describe the intended protection properties of the unit without yet prescribing the mechanisms used to achieve them.

Each objective must be associated with exactly one evaluation unit:

$$\forall O_{ij}, \exists!\, U_i$$

This uniqueness constraint prevents ambiguity in attachment and ensures that all subsequent refinements remain structurally anchored.

4.2.2 Level 4: security criterion

Each objective is refined into one or more structured criteria:

$$C_{ijk} \rightarrow O_{ij}$$

Security Criterion decomposes each objective into measurable assurance domains. While objectives define protection intent, criteria organize that intent into analyzable categories that prepare for requirement derivation.

For instance, the objective of preserving firmware integrity may be decomposed into criteria such as cryptographic authenticity validation, update control, and rollback prevention. Similarly, the objective of protecting cryptographic keys may yield criteria addressing storage security, access control enforcement, and lifecycle management.

Each criterion must derive from exactly one objective:

Table 4 Assurance-Oriented Hardware Decomposition

Step	Methodological Role	Description
Input	Scoped System of Interest	System boundary defined at Level 1 using the C-E-P model
Candidate Identification	Structural Enumeration	Identification of architectural elements, interfaces, and subsystems within scope
Structural Assessment	Assurance Semantics	Evaluation of candidates against assurance-relevant structural properties (trust transitions, exposure surfaces, evidential separability)
Rule Application	Deterministic Decomposition	Application of fixed split/merge rules derived from assurance logic, not design hierarchy
Closure	Structural Consistency	Iterative consolidation until no further partitioning or merging is justified
Output	Evaluation Units	Non-overlapping, structurally coherent units suitable for later assurance attachment

$\forall C_{ijk}, \exists! \, O_{ij}$

This constraint enforces hierarchical consistency and ensures that no assurance domain is introduced without a clearly articulated protection intent.

Through Levels 3 and 4, the framework transitions from abstract protection goals to structured assurance domains while maintaining explicit parent–child relationships. This prepares the model for normative operationalization at the requirements level.

4.3 Requirements

The Requirements segment introduces normative operationalization. Criteria are transformed into concrete, testable security requirements.

4.3.1 Level 5: security requirement

Security requirements translate structured criteria into concrete, enforceable controls. While objectives articulate protection intent and criteria organize domains of concern, requirements define the exact mechanisms by which those concerns are addressed. They must be clear, unambiguous, and testable, forming the direct foundation for verification.

Formally, each requirement R_{ijkl} derives from a single criterion C_{ijk} and is bound to one evaluation unit U_i:

$R_{ijkl} \rightarrow C_{ijk} \rightarrow O_{ij} \rightarrow U_i$

In addition, each requirement must be traceable to at least one authoritative normative source.

For example, under a firmware integrity criterion, a requirement may state:

> "*The firmware image shall be digitally signed using SHA-256 and validated against the public key stored in the secure element before execution.*"

This statement operationalizes the abstract objective of preserving firmware integrity into a concrete cryptographic control. It specifies the protection mechanism, the algorithmic constraint, and the validation condition in a form suitable for subsequent verification.

Different evaluation units yield distinct classes of requirements. Hardware-oriented units may emphasize tamper resistance, fault injection resilience, or side-channel mitigation. Firmware-oriented units may focus on secure boot enforcement, input validation, and authenticated update control. By tailoring requirements to the structural nature of each unit, the framework ensures that assurance remains both technically relevant and systematically structured.

Through this level, abstract protection intent becomes implementable security control, and each requirement remains hierarchically anchored within the framework.

4.4 Verification

The Verification level operationalizes the framework's analytic foundation by transforming requirements into observable and reproducible evidence. Whereas requirements prescribe what protection must exist, verification conditions define how that protection is demonstrated in practice.

4.4.1 Level 6: verification condition

For each requirement R_{ijkl}, at least one verification condition must exist:

$\forall R_{ijkl}, \exists V_{ijklm}$

Verification conditions define the observable and testable evidence required to confirm that a security requirement has been satisfied. Each condition specifies the circumstances under which a requirement is evaluated, the method of validation, and the form of acceptable evidence. In this way, verification conditions serve as the bridge between specification and assurance outcome.

For example, where a requirement mandates firmware signature validation before execution, the verification condition may state:

> *"During system boot, test logs must confirm that the firmware image signature is validated against the manufacturer's public key before code execution begins."*

Similarly, for a requirement on key storage security, a verification condition could require evidence from penetration testing or side-channel analysis demonstrating that secret keys cannot be extracted from the secure element under defined attack conditions.

By articulating verification conditions in this form, the framework ensures that each assurance claim is repeatable, transparent, and auditable. The resulting evidence enables independent evaluators to reproduce tests and confirm outcomes, thereby grounding assurance not in implicit trust but in demonstrable and verifiable evidence.

4.5 Hierarchical traceability constraint

To maintain structural coherence and end-to-end traceability, the framework enforces a strict hierarchical derivation across all assurance levels. Each assurance artifact must be derived from a parent artifact located at the immediately preceding level of the framework hierarchy. This rule ensures that assurance reasoning progresses systematically from

architectural scope to verifiable evidence while preserving an explicit lineage between system structure, protection intent, and validation activities.

Formally, the derivation structure is defined as:

$$U_i \rightarrow O_{ij} \rightarrow C_{ijk} \rightarrow R_{ijkl} \rightarrow V_{ijklm}$$

where U_i denotes an evaluation unit, O_{ij} denotes the j-th security objective associated with unit U_i, C_{ijk} denotes the k-th security criterion derived from objective O_{ij}, R_{ijkl} denotes the l-th security requirement derived from criterion C_{ijk}, and V_{ijklm} denotes the m-th verification condition validating requirement R_{ijkl}.

Through this hierarchical chain, evaluation units provide the architectural anchor for assurance reasoning, objectives articulate protection goals within each unit, criteria refine those goals into structured assurance domains, requirements translate the criteria into concrete security controls, and verification conditions define the observable evidence used to confirm their satisfaction.

The hierarchical derivation is governed by several structural constraints.

Constraint 1: Uniqueness of parent attachment

Uniqueness of parent attachment ensures that each artifact is associated with exactly one parent element at the immediately preceding level. Formally,

$$\forall x \in \mathcal{A}_L, !\, p \in \mathcal{A}_{L-1} \text{ such that } p \rightarrow x$$

This constraint guarantees structural clarity and prevents conflicting inheritance of assurance logic.

Constraint 2: Non skipping of levels

Sequential derivation requires that artifacts be derived strictly across adjacent levels of the hierarchy. Skipping intermediate levels is not permitted, ensuring that protection intent is refined through a disciplined progression from objectives to criteria and from criteria to requirements. Formally,

$$\neg\exists\left(U_i \rightarrow C_{ijk}\right), \neg\exists\left(O_{ij} \rightarrow R_{ijkl}\right), \neg\exists\left(C_{ijk} \rightarrow V_{ijklm}\right)$$

Constraint 3: Completeness of verification mapping

Completeness of verification mapping requires that every requirement be supported by at least one verification condition,

$$\forall R_{ijkl},\ \exists\, V_{ijklm}$$

thereby ensuring that all defined protections are operationalized through observable and testable evidence.

Constraint 4: Preservation of normative lineage

Preservation of normative lineage requires that requirements and verification conditions retain traceability to their originating normative sources, such as standards, regulatory guidance, or documented threat rationales. This constraint ensures that assurance claims remain auditable and defensible within regulatory and evaluation contexts.

These constraints ensure that assurance artifacts remain structurally anchored within the system architecture and that the reasoning process connecting security objectives to verification evidence remains transparent, reproducible, and methodologically consistent.

5 Evaluation

5.1 Evaluation methodology

This study adopts a Design Science Research (DSR) evaluation perspective [63], in which the proposed framework is assessed by applying it to a representative problem domain. In DSR, the goal of evaluation is not only to demonstrate that an artifact can be instantiated, but also to provide evidence of its utility, rigor, and relevance.

To achieve this, a case study evaluation approach was selected. Case studies are particularly suited for assurance frameworks because they allow for detailed analysis of how abstract structures (scope, objectives, requirements, verification) can be instantiated on real systems with complex assurance demands. Through systematic instantiation, the evaluation provides tangible outputs (e.g., scoped units, consolidated requirements, verification conditions) and enables comparative analysis against baseline practices.

The evaluation logic follows the layered structure of the framework (L1-L6), ensuring traceability from abstract objectives down to testable evidence. This structured alignment allows the framework's contributions to be assessed in terms of coverage, redundancy reduction, traceability, and contextual fitness.

5.2 Case study selection and rationale

To evaluate the framework under realistic and demanding conditions, a wireless fingertip pulse oximeter was selected as the case study. The device is an FDA-regulated medical product indicated for spot-check measurement of functional oxygen saturation (SpO_2) and pulse rate in adult and pediatric patients. It is deployed in both clinical and home healthcare environments. For confidentiality reasons, the specific manufacturer and model designation are not disclosed. The evaluation relies exclusively on publicly available documentation and observable characteristics. No proprietary design artifacts, firmware source code, or internal manufacturer information were accessed.

The selected device combines safety–critical physiological monitoring with wireless connectivity, creating a dual assurance challenge. Failures in sensing or signal interpretation may directly affect patient outcomes, while wireless communication introduces exposure to cyber-physical threats including interception, spoofing, downgrade, and integrity manipulation risks. This intersection of medical criticality and networked functionality makes the device representative of contemporary connected medical systems.

The system architecture comprises a sensing chain, embedded microcontroller and firmware subsystem, Bluetooth Low Energy (BLE) radio and protocol stack with authenticated encryption, power and battery management subsystem, physical enclosure, and update or service interface. These heterogeneous components impose diverse assurance obligations across measurement integrity, secure execution, communication protection, and physical robustness.

The device operates within a layered regulatory and standards landscape. Medical electrical safety and lifecycle processes impose obligations for development and risk management, while cybersecurity and cryptographic standards introduce additional, partially overlapping requirements. This fragmented normative environment provides a structured stress test for the framework's ability to consolidate heterogeneous standards, eliminate redundancy, preserve lineage, and link requirements to verification evidence without loss of contextual relevance.

The system architecture of the wireless fingertip pulse oximeter is shown in Fig. 4. The device boundary encloses the sensing chain, microcontroller and firmware (including secure boot and cryptographic primitives), BLE radio and stack, power and physical subsystem, user interface, and update/service port. External elements such as the mobile companion application, clinical/home environment, and cloud or EHR services are shown for context but lie outside the assurance scope of this study.

5.3 Evaluation procedure

The evaluation was conducted through a structured, stepwise procedure that instantiated the framework on the wireless fingertip pulse oximeter. The procedure followed the logical progression of the framework, beginning with system modeling and proceeding through categorization, requirements specification, and verification, before concluding with comparative assessment.

Step 1: System Modeling and Evaluation Unit Definition. The boundary-driven SoI model was applied to define the assurance scope (L1). The scope included the oximeter hardware, firmware, and wireless interface, as well as operational processes such as secure updates, while excluding external mobile applications and cloud services. Within this scope, the AHD method was employed to derive evaluation units (L2). Following the split/merge rules, four units were identified: U_{SENSE} (sensing and analog chain), U_{CPU} (compute and firmware), U_{BLE} (BLE communication), and U_{PHYS} (power and physical integrity). Each unit was documented through a Unit Contract, specifying its boundary, interfaces, assurance concerns, and evidence modalities.

Step 2: Definition of Security Objectives and Criteria. For each evaluation unit, high-level security objectives (L3) were defined to express what must be preserved or protected. These were then refined into structured security

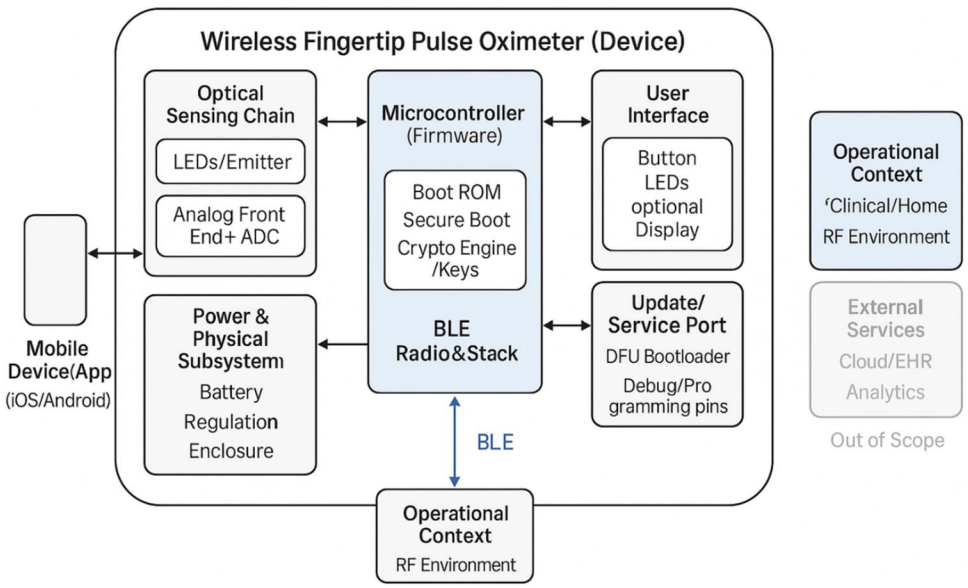


Fig. 4 The system architecture of the wireless fingertip pulse oximeter

criterion (L4) to ensure systematic coverage. Examples include maintaining firmware integrity and preventing rollback attacks for U_{CPU}, ensuring confidentiality and authenticity of wireless transmissions for U_{BLE}, and preserving measurement accuracy for U_{SENSE}. This step prevented assurance reasoning from remaining abstract and prepared a clear foundation for requirement derivation.

Step 3: Specification of Security Requirements

Explicit, testable security requirements (L5) were derived from the criteria. To ensure completeness, requirements were populated by consolidating multiple authoritative standards, including IEC 60601–1, IEC 62304, UL 2900–2-1, and FIPS 140–3. Overlaps and redundancies were reconciled while maintaining full traceability to all source standards. Each requirement was explicitly linked to its evaluation unit, ensuring contextual fitness and preserving authoritative lineage.

Step 4: Definition of Verification Conditions.

Verification conditions (L6) were defined to specify the evidence needed to confirm that each requirement was satisfied. For example, U_{CPU} requirements on firmware authenticity were linked to boot log inspection and key validation tests, U_{BLE} requirements on encryption were linked to packet capture analysis, and U_{PHYS} requirements on tamper resistance were linked to physical probing and power fault injection experiments. Verification conditions were defined with sufficient precision to allow reproducibility by independent evaluators.

Step 5: Comparative Assessment Against Baseline Standards.

Finally, the framework's outputs were compared against a baseline approach in which requirements were managed separately under individual standards. This comparison was used to evaluate the framework's contribution to coverage, redundancy reduction, traceability, and gap identification.

The baseline comparison was performed by independently mapping standards clauses without consolidation, documenting duplicate requirements and traceability gaps. The number of distinct firmware-related obligations was reduced from 7 isolated clauses to 3 consolidated requirements under the framework. This quantitative reduction illustrates the framework's structural efficiency while preserving complete normative lineage.

6 Results & analysis

The evaluation of the oximeter demonstrates how the proposed framework produces concrete, traceable outputs across all six levels (L1-L6). Beginning with system scoping and decomposition, the framework yields structured evaluation units, objectives, criteria, consolidated requirements, and verification conditions. These results provide evidence that the framework can systematically reconcile heterogeneous standards into a unified assurance structure.

To illustrate this in practice, the framework was instantiated on the wireless fingertip pulse oximeter following the procedure outlined in Sect. 5.3. This process yielded outputs at each architectural level, providing the basis for subsequent analysis. The evidence demonstrates that the framework can be applied systematically to a realistic device, producing reproducible artifacts rather than remaining at a conceptual level.

Assurance Scope (L1)

Using the C-E-P triad, the assurance scope was explicitly defined. Table 5 summarizes the included and excluded elements. This scoping step ensured that assurance activities were grounded in a clear boundary rather than relying on implicit assumptions.

Evaluation Units (L2)

Applying Assurance-Oriented Hardware Decomposition (AHD), the scoped system was partitioned into four evaluation units. Table 6 lists these units together with their primary assurance concerns and evidence modalities.

Security Objectives and Criteria (L3-L4)

From the defined units, high-level objectives were articulated and refined into criteria. Table 7 shows an excerpt from four evaluation units, illustrating how objectives were translated into systematic assurance categories.

Table 5 Assurance Scope (L1) for Wireless Fingertip Pulse Oximeter

Category	In Scope	Out of Scope
System Component	Optical sensing chain (LEDs, photodiode, AFE + ADC); MCU and firmware; BLE radio; power subsystem; enclosure	Mobile application; cloud services
System Environment	Local RF environment; USB/charging interface; clinical/home operating context	Remote networks; hospital IT infrastructure
System Process	Secure firmware update; device pairing; provisioning	Manufacturing supply chain

Table 6 Evaluation Units (L2) and Assurance Focus

Evaluation Unit	Description	Primary Assurance Concerns	Evidence Modalities
U_{CPU}	Microcontroller and firmware	Firmware authenticity; rollback prevention; runtime integrity	Boot logs; cryptographic validation; code review
U_{SENSE}	Optical sensing and analog chain	Measurement accuracy; signal integrity	Calibration tests; error analysis
U_{BLE}	BLE radio and stack	Confidentiality and authenticity of data; secure pairing	Packet capture; encryption/authentication checks
U_{PHYS}	Power subsystem and enclosure	Resistance to tampering; fault injection resilience	Penetration testing; fault injection logs

Table 7 Example objectives and criteria evaluation units

Evaluation Unit	Security Objective (L3)	Criteria (L4)
U_{CPU}	Preserve integrity of firmware at boot	• Cryptographic authenticity validation • Rollback prevention • Integrity of update mechanism
	Ensure confidentiality of cryptographic keys	• Secure key storage • Access control enforcement • Protection against leakage through debug interfaces
	Maintain runtime integrity of execution	• Memory protection (MPU enforcement) • Input validation against malformed data • Error handling without leakage
U_{SENSE}	Preserve accuracy of SpO_2 and pulse measurements	• Calibration against reference values • Filtering of ambient noise and artifacts • Secure transfer of raw data to MCU
	Ensure integrity of sensor data pipeline	• No bypass of AFE/ADC • Error detection in analog–digital conversion • Consistency checks between samples
	Protect against spoofing or manipulation	• Resistance to external optical interference Detection of abnormal light injection attempts
U_{BLE}	Preserve confidentiality of transmitted data	• End-to-end encryption (AES-CCM) • Key exchange via authenticated pairing • Protection against passive eavesdropping
	Ensure authenticity of data and endpoints	• Mutual authentication in pairing • Message authentication codes (MACs) • Replay attack prevention
	Maintain availability of wireless link	• Resistance to denial-of-service attempts • Robustness against jamming/interference
U_{PHYS}	Preserve device availability under power variation	• Brownout detection • Controlled shutdown on low battery • Fault-tolerant power regulation

Security Requirements (L5)

Requirements were derived from authoritative standards such as IEC 62304 (software lifecycle controls), UL 2900–2-1 (secure update mechanisms), and FIPS 140–3 (cryptographic key handling). These were harmonized into a coherent set that preserves lineage to the original sources while reducing redundancy and contradiction. The full consolidated requirement set is provided in Appendix A, representing the assurance obligations defined within the evaluation boundary. Representative excerpts are shown in Table 8, illustrating how high-level objectives and criteria are translated into concrete, testable requirements with explicit linkage to their originating standards, thereby ensuring both traceability and defensibility of the assurance process.

The reconciliation process can be illustrated through the firmware authenticity requirement under U_{CPU}. UL 2900-2-1 §5.2 requires authentication of firmware updates prior to acceptance, whereas FIPS 140-3 §7.6 mandates cryptographic integrity verification for approved modules. Although expressed at different abstraction levels, both clauses encode a shared assurance intent: firmware authenticity must be verified before execution or installation.

Within the framework, these overlapping clauses were consolidated into a single requirement with dual lineage:

Table 8 Representative excerpts of security objectives (L3) and criteria (L4) into concrete requirements (L5)

Security objective (L3)	Security criterion (L4)	Requirement (L5)	Referenced standard(s)/Act(s)
U_{CPU}- Preserve integrity of firmware at boot	Cryptographic authenticity validation	The device shall verify the digital signature of all firmware images using SHA-256 and RSA-2048 before execution	UL 2900-2-1 §5.2; FIPS 140-3 §7.6
	Rollback prevention	The device shall enforce monotonic version counters to prevent installation of outdated firmware	IEC 62304 §5.7.2; FDA Cybersecurity Guidance (2023)
	Integrity of update mechanism	Firmware updates shall only be accepted if validated against manufacturer-issued certificates	UL 2900-2-1 §6.3; IEC 62304 §5.8
U_{SENSE}- Ensure confidentiality of cryptographic keys	Secure key storage	Keys shall be stored only in secure hardware-protected registers, never exposed in system memory	FIPS 140-3 §7.7; HIPAA Security Rule §164.312
	Access control enforcement	Debug interfaces shall require authenticated access before reading protected areas	IEC 60601-1 §14.13; UL 2900-2-1 §6.4
	Leakage prevention	Test results must confirm that keys are not observable via debug ports or side channels	NIST SP 800-57; CWE-1247
U_{BLE}- Preserve confidentiality of transmitted data	End-to-end encryption	All BLE packets shall be encrypted and authenticated using AES-CCM with 128-bit keys	FIPS 140-3 §7.8; UL 2900-2-1 §5.4.2
	Secure key exchange	BLE pairing shall employ authenticated LE Secure Connections to prevent MITM	Bluetooth Core Spec v5.3; UL 2900-2-1 §5.4.3
	Replay attack prevention	Devices shall reject packets with reused nonces or counters	NIST SP 800-57; IEC 62304 §5.9.1
U_{PHYS}- Mitigate fault injection attacks	Power glitch resistance	The device shall detect and reset on abnormal voltage fluctuations	IEC 60601-1 §14.8; CWE-1247
	Clock manipulation detection	The MCU shall implement watchdog timers to detect abnormal clock frequencies	UL 2900-2-1 §5.6.4
	Key protection under faults	Cryptographic keys shall remain inaccessible under induced faults	FIPS 140-3 §7.9; NIST SP 800-160 Vol.2

R_CPU_BOOT_SIG. The device shall verify the digital signature of all firmware images using SHA-256 and RSA-2048 prior to execution.

The consolidation followed three principles. First, semantic equivalence was assessed at the level of assurance intent rather than surface phrasing. Second, where clauses differed in specificity, the stronger technical constraint was preserved in the merged requirement. Third, traceability to all originating standards was explicitly retained. In this manner, redundancy was eliminated without weakening normative strength or audit defensibility.

Beyond redundancy reconciliation, the framework also exposed coverage gaps in baseline standards. For example, UL 2900-2-1 specifies secure update authentication but does not explicitly address rollback prevention. The framework therefore introduced a monotonic version counter requirement, preserving lineage to UL 2900-2-1 and FDA Cybersecurity Guidance. Similarly, IEC 62304 emphasizes lifecycle controls but provides no explicit guidance on optical spoofing resistance within sensing subsystems; the framework extended coverage by defining spoof detection criteria under U_{SENSE}.

Verification Conditions (L6)

The final stage defines verification conditions (L6) that formalize the evidence required to demonstrate compliance with each security requirement. While requirements specify what protections must exist, verification conditions specify the observable, testable outcomes that confirm those protections are implemented correctly. This distinction prevents assurance from remaining declarative and anchors it in reproducible evaluation activities.

Each security objective (L3) is systematically decomposed into criteria (L4), requirements (L5), and verification conditions (L6), as illustrated in Table 9. The verification conditions are intentionally formulated as falsifiable conditions rather than descriptive statements. A requirement is considered satisfied only if its associated verification condition can be independently observed, tested, or audited.

The results presented above demonstrate that the framework generates structured and traceable assurance artifacts by consolidating requirements from heterogeneous standards and systematically linking them to explicit verification conditions. This structured mapping enhances coverage, reduces normative redundancy, and preserves end-to-end traceability across all architectural levels (L1-L6). At this stage, the contribution is structural and analytical: the framework establishes completeness, reconciliation, and operational testability, but it does not yet provide a mechanism for comparative measurement or prioritization across subsystems. Section 7 addresses this limitation by introducing a quantitative methodology for scoring and aggregation while preserving lineage to the underlying verification conditions.

Application of the framework to the oximeter case study further revealed assurance gaps that would remain unaddressed under a standards-only evaluation. For example, UL 2900-2-1 mandates authentication of firmware updates but does not explicitly require rollback prevention, thereby leaving devices potentially vulnerable to downgrade attacks. The framework identified this omission and formalized a monotonic version counter requirement, retaining traceable lineage to UL 2900-2-1 and FDA Cybersecurity Guidance. Similarly, IEC 62304 emphasizes lifecycle governance but does not address optical spoofing resistance within sensing subsystems. The framework extended coverage by defining explicit criteria for spoof detection under the U_{SENSE} unit. These examples illustrate that the framework performs analytical reconciliation rather than passive clause aggregation.

To make this distinction explicit, Table 10 contrasts a baseline standards-only evaluation with the framework outputs. The comparison demonstrates that the framework not only consolidates overlapping clauses but also reconciles redundancies and introduces explicit coverage where normative documents remain silent.

Beyond artifact generation, validation of the framework focused on three dimensions: (i) elimination of redundancy without loss of normative lineage, (ii) systematic exposure of coverage gaps not explicitly addressed by baseline standards, and (iii) demonstrable linkage between requirements and independently testable verification conditions. Each consolidated requirement was bidirectionally cross-mapped to its originating clauses to ensure that no normative constraint was omitted or weakened during reconciliation. This structured cross-validation supports the claim that the framework

Table 9 Representative excerpts of security objectives (L3) into criteria (L4), requirements (L5), and verification conditions (L6)

Security Objective (L3)	Security Criterion (L4)	Requirement (L5)	Verification Condition (L6)
U_{CPU}- Preserve integrity of firmware at boot	Cryptographic authenticity validation	The device shall verify the digital signature of all firmware images using SHA-256 and RSA-2048 before execution (UL 2900-2-1, FIPS 140-3)	Controlled boot testing must demonstrate that modified firmware images fail signature validation and prevent execution
U_{BLE}- Preserve confidentiality of transmitted data	End-to-end encryption	All BLE packets shall be encrypted and authenticated using AES-CCM with 128-bit session keys (FIPS 140-3, UL 2900-2-1)	Independent packet capture under active session must confirm encrypted payloads and authenticated session establishment; plaintext transmission shall not occur under any tested condition
U_{SENSE}- Ensure accuracy of SpO_2 measurements	Calibration integrity	Sensor calibration data shall be protected against unauthorized modification (IEC 60601-1, FDA Cybersecurity Guidance)	Attempted modification of calibration parameters without authenticated access must be rejected and logged; audit trail must confirm immutability
U_{PHYS}- Mitigate fault injection attacks	Power glitch resistance	The device shall detect and reset on abnormal voltage fluctuations (IEC 60601-1, CWE-1247)	Fault injection testing must demonstrate safe reset behavior with no execution bypass, cryptographic state exposure, or key material leakage

Table 10 Comparison of Standards-Only vs. Framework Outputs (Oximeter Case Study)

Evaluation Aspect	Standards-Only Approach	Framework Approach (This Paper)
Firmware update protection	Secure update authentication (UL 2900-2-1)	Consolidated requirement set including explicit rollback prevention (gap filled)
BLE confidentiality	Encryption requirement (FIPS 140-3)	Extended requirements covering replay attack prevention and entropy validation
Sensor accuracy & spoofing	Not explicitly covered in IEC 62304 / UL 2900	New assurance criteria for calibration integrity, noise filtering, and spoof detection
Requirement redundancy	Multiple overlapping clauses across UL 2900 and FIPS 140-3	Redundancies reconciled into a single coherent requirement set with preserved traceability
Traceability	Requirements scattered across separate documents	Structured end-to-end mapping: Objective → Requirement → Verification Condition

improves assurance coherence while maintaining regulatory defensibility.

7 Toward quantitative assurance

The preceding sections established a structured and traceable architecture for hardware security assurance, spanning scope definition, evaluation units, objectives, criteria, requirements, and verification conditions. While this layered framework enhances coherence and transparency, it also creates the necessary foundation for formal aggregation. Quantitative assurance in this context does not introduce probabilistic risk estimation; rather, it formalizes how structured verification evidence can be normalized and propagated across hierarchical levels.

This section develops the methodological basis for that extension, presenting the aggregation semantics, scoring principles, and illustrative instantiation that together enable disciplined comparative reasoning about assurance posture within the defined evaluation boundary.

7.1 Quantification objective and epistemic positioning

The preceding sections established a structured and traceable framework for hardware security assurance, spanning scope definition, evaluation units, objectives, criteria, requirements, and verification conditions. While this layered architecture improves transparency and coherence, it remains fundamentally qualitative unless supported by a formal aggregation logic. In practice, assurance stakeholders must make comparative judgments across subsystems, product versions, and lifecycle stages. Such decisions require more than narrative traceability; they require structured mechanisms for comparison and prioritization.

The objective of quantification in this work is therefore not to estimate probabilistic risk or to produce empirically calibrated security metrics. Rather, it is to formalize how structured assurance artifacts can be consistently aggregated into normalized indicators that preserve traceability across levels. The quantitative model introduced in this section provides a deterministic aggregation schema that maps verification outcomes to requirement-level, criterion-level, objective-level, unit-level, and system-level scores. This formalization enables evaluators to reason comparatively about assurance coverage while retaining explicit lineage to underlying evidence.

Importantly, the numerical values presented in this section should be interpreted as illustrative instantiations of the aggregation structure, not as empirically validated risk measurements. The framework defines the mathematical semantics of aggregation, including normalization constraints and weighting hierarchies, but it does not claim statistical calibration against real-world incident data. Calibration of scoring distributions and weight assignment mechanisms would require empirical datasets, structured expert elicitation, or regulatory benchmarking, and is therefore identified as future work.

The contribution of this quantitative extension lies in three aspects. First, it renders assurance aggregation explicit and reproducible at the structural level, eliminating implicit or ad hoc roll-up practices. Second, it enables diagnostic visibility by exposing how weaknesses at lower layers propagate to higher-level assurance outcomes. Third, it provides a quantification-ready foundation upon which empirical calibration or automated analytics may later be built.

By formalizing aggregation semantics without conflating them with probabilistic risk estimation, the framework preserves methodological rigor while avoiding overstatement of empirical validity. Quantification in this context should thus be understood as structured normalization and comparative reasoning grounded in traceable evidence, rather than as predictive security modeling.

7.2 Methodological building blocks

The proposed framework is inherently hierarchical, enabling quantitative evaluation through structured scoring and aggregation. For clarity, Table 11 summarizes the notation used throughout the aggregation model.

7.2.1 Verification scoring

Each verification condition $VC_{i,j,k}$ produces an observable outcome that can be scored. Let:

$$v_{i,j,k} \in [0,1]$$

denote the satisfaction value of verification condition k associated with requirement j of criterion i. Values may be binary (0 = fail, 1 = pass), ordinal (e.g., 0, 0.5, 1 for partial compliance), or probabilistic (confidence levels derived from statistical testing).

7.2.2 Requirement aggregation

Each security requirement $R_{i,j}$ consists of multiple verification conditions. Its score is defined as a weighted sum:

$$Score\left(R_{i,j}\right) = \sum_{k=1}^{m} \alpha_{i,j,k} \cdot v_{i,j,k}$$

where $\alpha_{i,j,k}$ are local weights for verification conditions, normalized such that:

$$\sum_{k=1}^{m} \alpha_{i,j,k} = 1$$

7.2.3 Criterion aggregation

Criteria C_i refine security objectives into structured assurance domains. The score for a criterion is:

$$Score\left(C_i\right) = \sum_{j=1}^{n} \beta_{i,j} \cdot Score\left(R_{i,j}\right)$$

with $\sum_{j=1}^{n} \beta_{i,j} = 1$. The weights $\beta_{i,j}$ capture the relative importance of requirements within the criterion, which may be informed by regulatory mandates, criticality to mission objectives, or known threat profiles.

7.2.4 Objective aggregation

At the objective level, scores consolidate criteria into higher-level protection goals:

$$Score(O) = \sum_{i=1}^{p} \gamma_i \cdot Score\left(C_i\right)$$

With normalization $\sum_{i=1}^{p} \gamma_i = 1$. Objectives thus yield quantifiable indicators of whether high-level security intentions (e.g., firmware integrity, confidentiality of data) are achieved.

7.2.5 Evaluation unit aggregation

Each evaluation unit U_q encompasses several objectives. The unit-level assurance score is:

$$Score\left(U_q\right) = \sum_{r=1}^{s} \delta_{q,r} \cdot Score\left(O_r\right)$$

With normalization $\sum_{r=1}^{s} \delta_{q,r} = 1$ where $\delta_{q,r}$ are weights reflecting the relative importance of objectives within the unit.

Crucially, weights should also be assigned at the evaluation unit level to reflect differences in subsystem criticality. For instance, in a medical oximeter, compromise of the sensing chain (U_{SENSE}) directly affects patient safety, whereas compromise of the BLE interface (U_{BLE}) primarily affects confidentiality. Thus, evaluation units are assigned criticality weights λ_q, normalized across all units:

Table 11 Notation used in the hierarchical aggregation model

Symbol	Meaning	Domain/Constraint
$v_{i,j,k}$	Satisfaction score of verification condition k for requirement j under criterion i	[0, 1]
$\alpha_{i,j,k}$	Weight of verification condition within requirement	$\sum_k \alpha_{i,j,k} = 1$
$S(R_{i,j})$	Score of requirement j under criterion i	[0, 1]
$\beta_{i,j}$	Weight of requirement within criterion	$\sum_j \beta_{i,j} = 1$
$S(C_i)$	Score of criterion i	[0, 1]
γ_i	Weight of criterion within objective	$\sum_i \gamma_i = 1$
$S(O_r)$	Score of objective r	[0, 1]
$\delta_{q,r}$	Weight of objective within evaluation unit q	$\sum_r \delta_{q,r} = 1$
$S(U_q)$	Score of evaluation unit q	[0, 1]
λ_q	Criticality weight of evaluation unit q	$\sum_q^Q \lambda_q = 1$
$S(System)$	Aggregated system-level assurance score	[0, 1]

$$\sum_{q=1}^{Q} \lambda_q = 1$$

7.2.6 System-level aggregation

Finally, the system-level assurance score is obtained as a weighted sum of unit scores:

$$Score(System) = \sum_{q=1}^{Q} \lambda_q \cdot Score\left(U_q\right)$$

This hierarchical aggregation maintains traceability from individual verification conditions to the overall system assurance posture.

7.2.7 Rationale for weighting

Weights at each level $(\alpha, \beta, \gamma, \delta, \lambda)$ may be determined by one or more of the following rationales:

- Standards-based weighting: Certain requirements are explicitly prioritized in authoritative standards (e.g., FIPS 140-3 Level 3 physical tamper resistance). The challenge is that standards often differ in emphasis across domains, and direct mapping may over-privilege compliance obligations while underrepresenting emerging threats.
- Risk-based weighting: Weights can reflect the impact of a requirement's failure, derived from likelihood × consequence analysis. This approach aligns with established risk management practices but depends on accurate threat modeling and reliable data, which are often costly or incomplete.
- Expert judgment: Domain experts or regulators can assign weights based on experience. While practical, this method is inherently subjective and can be influenced by organizational biases or shifting regulatory priorities, limiting reproducibility.
- Uniform weighting: In exploratory phases, assigning equal weights preserves neutrality and avoids subjective disputes. However, this simplicity can mask genuine differences in criticality across requirements or subsystems.

In practice, no single approach is sufficient. A hybrid strategy is often necessary: standards may define minimum priorities, risk assessments can refine criticality, expert elicitation can fill data gaps, and uniform weighting can serve as a baseline for sensitivity analysis. By combining these perspectives, weight assignments can become both more defensible and more adaptable to diverse assurance contexts.

7.2.8 Sensitivity considerations

Because the weighting parameters (α, β, γ, δ, λ) are used to illustrate the aggregation methodology rather than represent empirically calibrated values, the stability of the resulting system score should be interpreted cautiously. To examine the robustness of the diagnostic insights produced by the framework, we conducted a limited sensitivity check by varying the weighting parameters within a small range (± 10%) while preserving their relative ordering. Across these variations, the relative ranking of subsystem assurance levels remained consistent, indicating that the framework's diagnostic conclusions are driven primarily by structural evidence coverage rather than by specific weight assignments.

A comprehensive empirical calibration of these parameters across multiple hardware systems remains an important direction for future research.

7.3 Verification scoring rubric and worked example

7.3.1 Verification scoring rubric

The aggregation model introduced in Sect. 7.2 requires assignment of normalized satisfaction values $v_{i,j,k} \in [0,1]$ to verification conditions. Without structured guidance, such assignments risk becoming inconsistent across evaluators, reducing reproducibility and comparability. To mitigate this, a constrained scoring rubric is introduced to standardize interpretation of verification outcomes while preserving necessary expert judgment.

The objective of the rubric is not to impose artificial precision, but to anchor scoring decisions within clearly defined qualitative bands linked to evidentiary strength and residual assurance impact.

7.3.2 Scoring scale definition

Table 12 defines five recommended scoring levels used to represent the degree to which a verification condition is satisfied. The scale is intentionally limited to a small number of ordinal bands in order to balance interpretability and discrimination power while avoiding excessive granularity that could create a misleading sense of numerical precision.

The five-level scale was selected for three reasons:

It aligns with common assurance maturity models that use ordinal banding rather than continuous probability.
It provides sufficient resolution to distinguish minor deviations from structural weaknesses.
It limits overfitting and artificial score differentiation.

Table 12 Verification scoring rubric used for evaluation of verification conditions

Score	Interpretation	Typical Evidence	Assurance Impact
1.0	Fully satisfied	Control verified under adversarial or stress testing; no deviation observed	No residual weakness
0.9	Minor deviation	Minor implementation issue or edge-case anomaly without exploitability	Low residual concern
0.7	Moderate weakness	Partial coverage; mitigation present but incomplete; exploit unlikely but plausible	Medium residual concern
0.5	Significant gap	Control partially implemented or inconsistently enforced; realistic exploitation scenario exists	High residual concern
0.0	Failed	Control absent, ineffective, or bypassable under realistic conditions	Critical weakness

Scores should be assigned based on documented evidence, such as adversarial testing results, fuzzing outcomes, coverage metrics, or formal validation artifacts.

To improve consistency across evaluators, the scoring levels can be interpreted according to the qualitative rubric summarized in Table 12. A score close to 1.0 represents full compliance with the requirement supported by verified evidence and no identified weaknesses. Scores around 0.9 indicate minor implementation deviations that do not materially affect the underlying security objective. Scores around 0.7 represent moderate weaknesses or incomplete verification coverage that may increase residual risk under certain threat conditions. Values near 0.5 indicate partial implementation or significant gaps in supporting evidence. Scores below 0.5 reflect major deficiencies or missing protection mechanisms. These ranges should be interpreted as guidance rather than exact numerical thresholds. The purpose of the rubric is to support structured and transparent evaluation reasoning by linking verification evidence to consistent satisfaction levels across different evaluators.

To further support consistent scoring, evaluations may follow a structured reconciliation procedure when multiple assessors review the same verification condition. Scores are initially assigned independently based on the available evidence. If the resulting scores fall within the same rubric band, the higher score may be retained provided that the supporting evidence is documented. When scores fall into different bands, evaluators conduct a reconciliation step in which the underlying evidence (e.g., adversarial testing results, fuzzing reports, coverage metrics, or formal verification artifacts) is jointly reviewed and the final score is aligned with the rubric definitions presented above. This procedure helps ensure that satisfaction values reflect documented verification evidence rather than individual interpretation. Because these satisfaction values serve as inputs to the hierarchical aggregation model described in Sect. 7.3, consistent scoring is essential for ensuring that subsystem-level verification outcomes can be reliably consolidated into system-level assurance indicators.

7.3.3 Calibration and reproducibility considerations

While expert judgment remains necessary, the rubric constrains interpretation by linking score bands to observable characteristics of verification outcomes. Evaluators should apply consistent criteria within an evaluation cycle and document justification for non-maximal scores.

Where multiple evaluators are involved, inter-review alignment sessions or consensus scoring may be used to reduce subjective divergence. Future work may formalize calibration through structured expert elicitation or empirical benchmarking against historical defect data.

Importantly, the scoring framework is designed to enhance structural reproducibility, that is, consistency in aggregation logic and interpretive banding, rather than to guarantee statistical reproducibility across independent organizations.

7.4 Worked example and system-level aggregation

To illustrate the hierarchical aggregation mechanism, the quantitative model is instantiated for the wireless fingertip pulse oximeter introduced earlier. All values are normalized to the interval [0, 1], and weights at each level are normalized as specified in Sect. 7.2.

7.4.1 Evaluation-unit criticality weights

As shown in Table 13, each evaluation unit is assigned a criticality weight based on its relative impact on patient safety and security. These λ_q values are primarily risk-based, prioritizing safety impact, but they could alternatively be derived from standards or expert elicitation. The assigned weights ensure that subsystem evaluations contribute proportionally to the aggregated system-level assurance score.

7.4.2 *Detailed roll-up for one unit (U_{CPU})*

The U_{CPU} evaluation unit of the oximeter is responsible for enforcing trusted boot and maintaining runtime execution integrity. In accordance with the L3-L6 hierarchy,

Table 13 Unit-level criticality weights (λ_q) and rationale

Evaluation Unit	Rationale (summary)	λ_q
U_{SENSE}	Direct effect on clinical accuracy (SpO_2), safety–critical	0.35
U_{CPU}	Trusted boot & runtime integrity underpin all controls	0.30
U_{PHYS}	Tamper/faults can cause unsafe behavior or bypasses	0.20
U_{BLE}	Primary confidentiality/authenticity channel	0.15
Total		1.00

two security objectives are considered: firmware integrity at boot and integrity preservation during runtime. Each objective contains explicit criteria, which are further decomposed into requirements and verification conditions. Aggregation at each level follows the weighting rules defined in Sect. 7.2.

A. **Security Objective O_1: Firmware Integrity**

This objective ensures that firmware authenticity is verified during system initialization and that downgrade attacks are prevented.

Criterion C_{boot}: Secure Boot Enforcement

This criterion addresses authentication and rollback resistance mechanisms.

Requirement R_1: Signature validation

The verification conditions are scored as 1.0 and 0.90, with weights (0.6, 0.4). The perfect score reflects tests confirming correct signature validation in all cases, while the slightly lower score represents a minor flaw observed under certain reset conditions.

$$Score(R_1) = 0.6 \cdot 1.0 + 0.4 \cdot 0.9 = 0.96$$

Requirement R_2: Rollback prevention

Verification conditions are scored as 1.0 and 0.50, with equal weights (0.5,0.5). The perfect score reflects a robust version counter mechanism, while the lower score represents partial coverage in testing, where rollback protection was bypassed in one simulated update sequence.

$$Score(R_2) = 0.5 \cdot 1.0 + 0.5 \cdot 0.5 = 0.75$$

Criterion aggregation:

Aggregating both requirements with weights $\beta = (0.7, 0.3)$ yields:

$$Score(C_{boot}) = 0.7 \cdot 0.96 + 0.3 \cdot 0.75 = 0.897$$

B. **Security Objective O2: Runtime Execution Integrity**

The second objective focuses on maintaining runtime execution integrity through memory protection and input validation.

Criterion $C_{runtime}$: Runtime Protection Mechanisms

This criterion evaluates memory isolation and input handling robustness.

Requirement R_3: MPU Enforcement

Verification condition scores are 0.80 and 0.7, with equal weights (0.5,0.5):

$$Score(R_3) = 0.5 \cdot 0.8 + 0.5 \cdot 0.7 = 0.75$$

Requirement R_4: Input validation (fuzzing)

Verification condition scores are 0.60 and 0.40, with weights (0.6, 0.4):

$$Score(R_4) = 0.6 \cdot 0.6 + 0.4 \cdot 0.4 = 0.52$$

Criterion aggregation

Aggregating both requirements with weights $\beta = (0.6, 0.4)$ gives:

$$Score(C_{runtime}) = 0.6 \cdot 0.75 + 0.4 \cdot 0.52 = 0.658$$

Unit Aggregation

Finally, the two objectives (boot integrity and runtime integrity) are combined at the unit level with weights $\delta = (0.65, 0.35)$, reflecting the higher criticality of secure boot relative to runtime controls.

$$Score(U_{CPU}) = 0.65 \cdot 0.897 + 0.35 \cdot 0.658 = 0.81335 \approx 0.813$$

Interpretation. The evaluation unit U_{CPU} achieves a normalized assurance score of approximately 0.813. The strong performance in the secure boot criterion (0.897) compensates partially for weaker runtime protection (0.658). The hierarchical decomposition enables transparent identification of improvement priorities. In this instance, strengthening runtime input validation and memory isolation mechanisms would most effectively increase the unit-level assurance score.

This quantitative roll-up preserves traceability from verification conditions to system-level representation, while enabling targeted diagnostic reasoning without conflating structural assurance with probabilistic risk estimation.

System-Level Score

The overall system-level assurance score is obtained by aggregating evaluation-unit scores using the criticality

weights λ_q defined in Sect. 7.3.1. Following the hierarchical aggregation model introduced in Sect. 7.2.

$$\begin{aligned} Score(System) &= \sum_q^Q \lambda_q \cdot Score\left(U_q\right) \\ &= 0.35 \cdot 0.8551 + 0.30 \cdot 0.81335 \\ &\quad + 0.15 \cdot 0.8607 + 0.20 \cdot 0.7610 \\ &= 0.8246 \approx 0.825 \end{aligned}$$

where Q denotes the total number of evaluation units included in the aggregation.

Interpretation. The system-level value represents the weighted aggregation of assurance coverage across all evaluation units within the defined scope. It should be interpreted as a normalized structural assurance indicator within the model, not as a probabilistic estimate of system compromise.

The hierarchical decomposition preserves full traceability. Stakeholders may drill down from the system score to individual evaluation units (e.g., $U_{PHYS} = 0.761$), and further to specific objectives, criteria, requirements, and verification conditions. This traceable structure enables targeted improvement planning. In the present case, strengthening physical protection controls would yield the most direct impact on the overall assurance score.

By maintaining alignment between quantitative aggregation and structural decomposition, the model supports comparative reasoning while retaining explicit linkage to underlying verification evidence.

Sensitivity Analysis of Weighting Stability

The aggregation model introduced in Sect. 7.2 employs normalized weights at multiple hierarchical levels, including the evaluation-unit criticality factors λ_q. While these weights are risk-informed, they inevitably reflect modeling assumptions. To assess the stability of the system-level conclusions under reasonable perturbations, a sensitivity analysis was conducted focusing on the evaluation-unit weights. The objective of this analysis is not statistical calibration, but robustness verification: specifically, whether moderate changes in weighting assumptions alter the diagnostic ranking of evaluation units.

7.4.3 ±10% Criticality Perturbation

Each evaluation-unit weight λ_q was varied independently by ± 10%, followed by renormalization to preserve the constraint:

$$\sum_q^Q \lambda_q = 1$$

For example, the baseline weight for U_{PHYS} is 0.20. Under perturbation:

- Lower bound: 0.18
- Upper bound: 0.22

After renormalization, the system-level score varied within the interval:

$$S_{sys} \in [0.816, 0.832]$$

The maximum deviation from the baseline value (0.825) was 0.009.

Importantly, under all perturbation scenarios:

- U_{PHYS} remained the lowest-scoring unit.
- The relative ranking of evaluation units was unchanged.

This indicates that moderate variation in criticality assumptions does not alter the identification of the weakest subsystem.

7.4.4 Uniform weighting scenario

To further assess structural stability, a uniform weighting configuration was applied:

$$\lambda_q = \frac{1}{4}, \forall q$$

Under this configuration, the recalculated system-level score was:

$$S_{sys}^{uniform} = 0.823$$

Although the absolute value differs slightly from the baseline, the ranking of unit-level scores remains:

$$U_{SENSE} > U_{BLE} > U_{CPU} > U_{PHYS}$$

The persistence of ranking under equal-weight conditions demonstrates that the diagnostic signal is driven primarily by unit-level performance rather than by weighting bias.

7.4.5 Rank stability test

To evaluate ordering robustness more formally, pairwise comparisons of unit scores were examined across all tested perturbation scenarios.

Let:

$$Score\left(U_a\right) > Score\left(U_b\right)$$

denote the baseline ordering relation. Across all perturbations tested:

$\text{sign}(Score(U_a) - Score(U_b))$ remained invariant for all unit pairs.

Thus, no rank inversions occurred under $\pm 10\%$ weight variation or uniform weighting. This stability suggests that the aggregation model's diagnostic conclusions are structurally robust to moderate modeling uncertainty.

7.4.6 Interpretation

The sensitivity results support three observations:

- Magnitude Stability: Absolute system scores exhibit limited fluctuation under reasonable perturbations.
- Ordering Stability: The weakest unit remains consistently identifiable.
- Diagnostic Reliability: Improvement prioritization recommendations are not artifacts of specific weight assignments.

It should be emphasized that this analysis does not claim statistical robustness under arbitrary weight configurations. Rather, it demonstrates that within plausible modeling bounds, the aggregation structure yields stable and reproducible diagnostic outcomes. This strengthens the methodological validity of the quantitative extension by confirming that the framework's conclusions are not unduly sensitive to minor weighting assumptions.

7.5 Conceptual and empirical status of the quantitative model

The quantitative extension presented in this section formalizes the aggregation semantics of assurance artifacts within the proposed framework. Clarifying its epistemic status is essential to avoid conflating structural normalization with empirical risk estimation.

The model constitutes a methodological aggregation structure rather than a statistically calibrated predictor of security outcomes. Its equations specify how verification evidence is normalized and propagated across hierarchical levels, ensuring internal consistency, traceability, and transparency. By explicitly encoding weighting and normalization constraints, the framework replaces implicit or ad hoc roll-up practices with reproducible aggregation logic.

The numerical values introduced in the case study serve as illustrative instantiations of this structure. They demonstrate operational feasibility and traceable computation but do not claim generalizable empirical validity across product classes, industries, or threat environments. Score assignments remain dependent on documented verification evidence within the defined evaluation boundary.

Empirical calibration of scoring distributions and weighting parameters represents a subsequent research trajectory. Such calibration could draw on structured expert elicitation, cross-system benchmarking, historical defect data, or longitudinal field observations. Importantly, meaningful calibration presupposes a stable and transparent aggregation structure. The primary contribution of this work lies in establishing that structural foundation.

Accordingly, the quantitative model should be understood as quantification-ready: it provides the formal mechanics necessary for reproducible normalization and diagnostic analysis, while deferring empirical parameter optimization to future validation efforts.

By distinguishing structural formalization from empirical calibration, the framework preserves methodological rigor without implying predictive validity beyond its defined scope.

7.6 Analytical applications in assurance evaluation

Sections 7.2 through 7.5 established the formal aggregation model, scoring semantics, and robustness properties of the proposed framework. Building on this foundation, the present section examines the analytical implications of the normalized assurance indicators produced by the model. The objective is not to introduce additional quantitative mechanisms, but to demonstrate how the structured aggregation enables systematic diagnostic reasoning within the defined scope.

Because normalization, weighting, and hierarchical traceability are explicitly encoded in the model, assurance results can be examined at multiple structural levels without losing linkage to underlying verification evidence. This permits disciplined analysis of assurance distribution, variability, and concentration across subsystems and responsibility domains.

The analytical capabilities are organized into four interconnected dimensions: intra-unit diagnostic decomposition, cross-unit distribution analysis, dependency concentration across organizational boundaries, and system-level resilience modeling. Together, these dimensions illustrate how the aggregation framework supports comparative reasoning and targeted improvement planning while preserving structural transparency.

7.6.1 Unit-level diagnosis

At the evaluation-unit level, normalized assurance scores can be decomposed to objective and criterion granularity. This hierarchical visibility enables identification of structural imbalance within a subsystem without losing linkage to underlying verification evidence.

For example, within U_{SENSE}, objective-level results reveal a divergence between SpO_2 Accuracy (0.88) and Sensor Pipeline Integrity (0.72), corresponding to an internal differential of 0.16. Such divergence signals uneven assurance maturity across protection goals within the same subsystem. Criterion-level inspection further localizes this imbalance: Calibration Integrity (0.95) exhibits strong coverage, whereas Secure Transfer (0.84) and Filtering Validation (0.85) contribute comparatively lower assurance levels.

As shown in Fig. 5, the mapping of criterion scores to their parent objectives reveals structured clustering rather than random variability. The dispersion observed at the objective level is therefore attributable to identifiable control domains rather than stochastic fluctuation.

This hierarchical decomposition supports two complementary forms of analytical reasoning. First, it enables localized identification of weaker assurance elements even when aggregate objective scores remain within acceptable bounds. Second, it supports targeted improvement planning by allowing mitigation efforts to focus on structurally weaker criteria rather than uniformly redistributing assurance resources.

It is important to emphasize that this analysis does not estimate exploit likelihood or attack probability. Rather, it quantifies internal assurance dispersion within the defined evaluation boundary, enabling disciplined diagnostic reasoning grounded in traceable aggregation semantics.

7.6.2 Cross-unit distribution analysis

Cross-unit comparison provides insight into how assurance maturity is distributed across the architectural components of the system. Figure 6 presents boxplots of objective-level scores for U_{CPU}, U_{SENSE}, U_{PHYS}, and U_{BLE}, reporting medians, interquartile ranges (IQR), and dispersion.

A threshold value of 0.75 is indicated to represent the minimum acceptable assurance level for objectives within the defined evaluation boundary. This threshold is not a regulatory mandate but an analytical reference point for comparative interpretation.

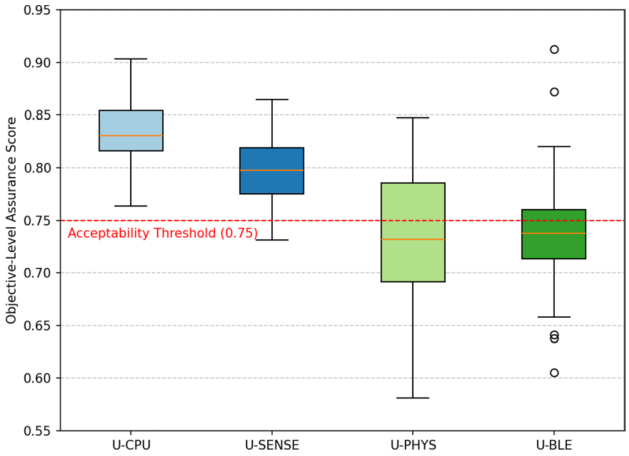


Fig. 6 Distribution of assurance scores across four evaluation units

The results reveal distinct distributional patterns across evaluation units. U_{CPU}(median 0.84, IQR 0.08) and U_{SENSE} (median 0.80, IQR 0.07) exhibit relatively narrow dispersion, indicating consistent objective-level performance and comparatively uniform control coverage. In contrast, U_{PHYS} (median 0.73, IQR 0.15) and U_{BLE}(median 0.74, IQR 0.14) display wider variability, with several objectives falling below the reference threshold.

The analytical significance lies not solely in median values but in dispersion magnitude. Larger interquartile ranges signal uneven assurance maturity within a subsystem, reflecting variability in control implementation or verification strength across objectives. Such variability may indicate localized structural weaknesses even when aggregate unit scores appear moderate.

Cross-unit analytics therefore enables identification of two distinct phenomena: absolute assurance deficits,

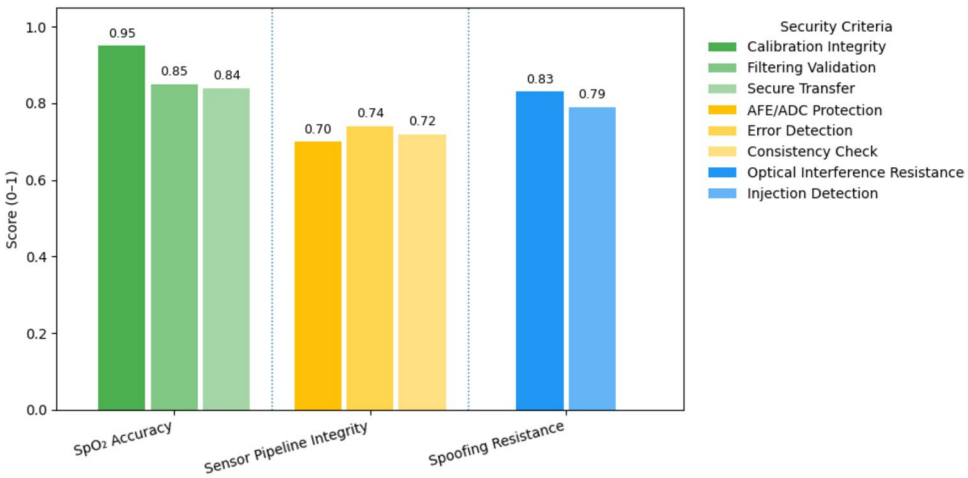


Fig. 5 Unit-level diagnosis figure for U_{SENSE}

reflected in lower central tendency, and assurance volatility, reflected in dispersion. The latter is particularly important, as high variability may signal fragility or inconsistent implementation across control domains.

7.6.3 Supply chain analytics

Beyond the distribution of technical assurance across subsystems, the framework enables structured modeling of assurance dependency across organizational responsibility domains. Within the defined technical verification boundary, each evaluation unit is associated with a governance category indicating whether the subsystem is primarily vendor-managed or supplier-provided. These categorizations do not alter the verification scope; rather, they provide an analytical dimension for examining how system-level assurance depends on components under different organizational control regimes.

Dependency concentration is quantified by weighting each evaluation unit's normalized score by its criticality factor λ_q, followed by renormalization within each organizational category. This procedure yields the relative share of total system-level assurance attributable to vendor-controlled and supplier-provided components (Appendix B shows the worked calculation for all units).

As illustrated in Fig. 7, vendor-controlled assurance is distributed between U_{CPU} (52.5%) and U_{PHYS} (47.5%), indicating relatively balanced internal contribution. In contrast, supplier-provided assurance exhibits stronger concentration, with U_{SENSE} accounting for 59.4% and U_{BLE} for 40.6%. The elevated contribution of U_{SENSE} reflects both its higher criticality weight and its structural role within the system architecture.

This asymmetry indicates concentration of assurance dependency within the supplier domain. From a governance perspective, assurance deficiencies within U_{SENSE} would therefore exert a disproportionate influence on the supplier-controlled assurance share and, consequently, on the aggregated system-level score. Such insights help evaluators identify subsystems where assurance risks may propagate through external dependencies.

It is important to distinguish this analytical modeling from direct evaluation of supply-chain processes. Upstream manufacturing workflows, fabrication controls, and supplier quality management systems remain outside the technical verification boundary defined in Sect. 5. The present analysis does not assess process maturity or supplier governance practices; rather, it quantifies structural reliance on externally controlled components using the same normalized aggregation semantics applied throughout the framework. Because these computations derive directly from unit-level scores and criticality weights, category-level results remain fully traceable to the underlying objectives, criteria, requirements, and verification conditions. This traceability ensures that governance-oriented insights are grounded in documented technical evidence rather than abstract attribution.

7.6.4 System-level resilience modeling

At the highest level of aggregation, the framework produces a normalized system-level assurance indicator derived from the criticality-weighted contributions of each evaluation unit. This aggregation reflects the structural distribution of assurance across subsystems within the defined scope.

Figure 8 presents this relationship as a waterfall visualization, where each unit contributes incrementally according to its weighted score $\lambda_q \cdot Score(U_q)$. The representation makes explicit the relative influence of each subsystem on the aggregated result. In the present case, U_{SENSE} and U_{BLE} contribute comparatively larger increments, while U_{PHYS} contributes less proportionally, consistent with its lower normalized score.

Residual risk is defined structurally as the complement of the system-level score:

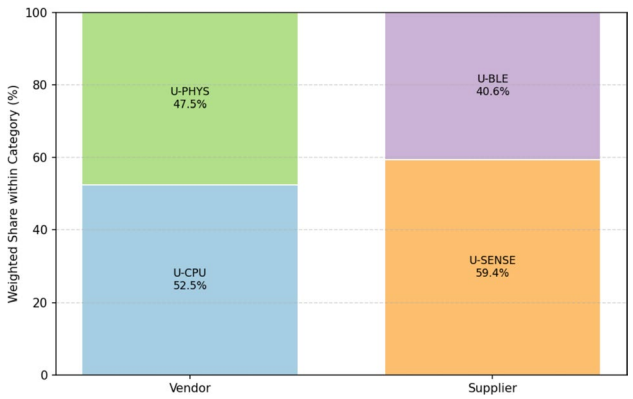


Fig. 7 Supply-chain assurance contribution distribution

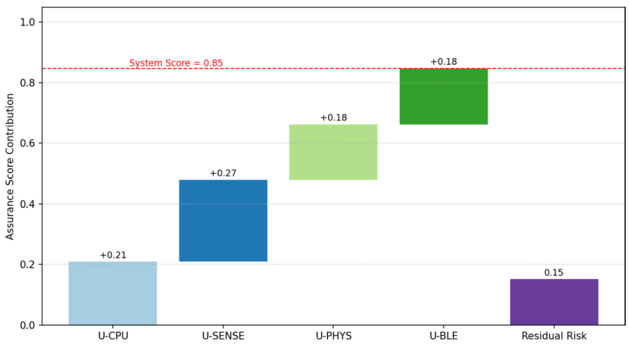


Fig. 8 System-level waterfall: Unit contributions and residual risk

$$ResidualRisk = 1 - S_{sys}$$

This quantity should not be interpreted as a probabilistic estimate of compromise. Rather, it represents the normalized assurance deficit relative to the modeled criteria set, that is, the distance from full satisfaction of the defined assurance objectives under the specified weighting scheme.

The waterfall decomposition supports two analytical functions. First, it provides contribution transparency by explicitly exposing how subsystem performance shapes the aggregate outcome. Second, because aggregation is linear and traceable, hypothetical improvements at the unit level can be propagated forward to estimate their effect on the overall system score, enabling structured prioritization of mitigation efforts.

By combining normalized aggregation with hierarchical traceability, the framework supports system-level reasoning while preserving explicit linkage to underlying verification evidence. It therefore functions as a structured decision-support mechanism rather than as a probabilistic risk model.

8 Discussion

The evaluation demonstrates that the proposed framework delivers structured, traceable, and actionable assurance artifacts, validating its ability to reconcile heterogeneous standards into a unified model. Compared to a standards-only baseline, the framework improves traceability, reduces redundancy, and establishes clear pathways from objectives to verification. Importantly, it also demonstrates readiness for quantitative evaluation, thereby bridging the gap between compliance documentation and measurable assurance posture.

8.1 Contributions and insights

The evaluation yields three principal insights regarding the contribution of the proposed framework.

First, the framework operationalizes assurance through a structured multi-level architecture (L1-L6) that systematically translates high-level protection objectives into verifiable technical artifacts. In the oximeter case, this progression encompassed boundary definition (L1), decomposition into evaluation units (L2), articulation of objectives and criteria (L3-L4), consolidation into 22 structured requirements (L5), and explicit verification conditions (L6). This layered structure demonstrates that abstract assurance concerns, such as firmware integrity or wireless confidentiality, can be rendered into traceable and enforceable technical controls, including signature validation procedures and cryptographic log inspection.

Second, the framework reconciles heterogeneous standards into a coherent and non-redundant requirement structure while preserving lineage to authoritative sources. Rather than treating standards as isolated compliance checklists, the model consolidates overlapping clauses into unified requirements with explicit traceability. For example, firmware update protections drawn from UL 2900–2-1 and FIPS 140–3 were integrated into a consolidated requirement addressing signature validation and rollback prevention, with cross-standard references maintained. This approach reduces fragmentation while strengthening coherence and audit defensibility.

Third, the framework embeds verification linkage directly within the assurance structure. Requirements are not merely cataloged; they are systematically paired with observable verification conditions. In practice, this linkage connects cryptographic requirements to inspectable entropy sources and implementation logs, and maps tamper detection obligations to observable system events. By explicitly connecting normative intent to technical evidence, the framework narrows the gap between assurance specification and assurance demonstration.

These contributions illustrate that structured consolidation, hierarchical traceability, and verification integration can be unified within a single assurance architecture. The result is not merely improved documentation coherence, but a foundation capable of supporting disciplined analysis and quantitative extension.

8.2 Practical implications

The framework carries several practical implications for stakeholders engaged in assurance activities within safety–critical and security-sensitive domains.

For regulators and auditors, the structured multi-level architecture provides traceable linkage from high-level assurance objectives to concrete verification conditions. Because requirements are explicitly mapped to authoritative standards and paired with observable evidence, certification and compliance arguments can be supported by transparent lineage rather than fragmented documentation. This structural traceability strengthens audit defensibility and reduces ambiguity in compliance interpretation.

For industry stakeholders managing heterogeneous regulatory obligations, the framework offers a systematic method for consolidating overlapping standards into a coherent requirement structure. By integrating clauses from multiple sources into unified, non-redundant requirements with preserved lineage, the approach reduces administrative complexity while maintaining regulatory accountability. This consolidation mitigates duplication of effort and clarifies cross-domain obligations.

For system designers and assurance managers, the framework enhances diagnostic visibility. Hierarchical decomposition and quantitative-ready aggregation enable identification of uneven assurance maturity across subsystems without losing linkage to underlying verification evidence. In the oximeter case, variation among evaluation units revealed differential coverage across sensing, communication, and physical protection domains, illustrating how structured aggregation can inform prioritization decisions. Rather than reallocating resources uniformly, assurance investment can be directed toward structurally weaker components identified through traceable analysis.

Finally, because the framework produces normalized and comparable indicators, it provides a basis for integrating assurance outputs into broader governance and decision-support processes. While not a substitute for formal risk modeling, the quantitative-ready structure supports comparative reasoning in contexts such as lifecycle planning, procurement assessment, and system-level review, where transparent and structured indicators are increasingly required.

8.3 Comparison to existing approaches

Positioning the proposed framework within the broader assurance landscape requires comparison with four related streams of work: standards-based assurance practices, assurance case methodologies, quantitative scoring frameworks, and hardware-focused evaluation models. Each contributes important elements to assurance engineering, yet each also exhibits structural limitations that motivate the integrated approach presented in this study.

8.3.1 Standards-based assurance

Standards such as FIPS 140-3, IEC 62304, ISO 14971, UL 2900-2-1, and ISO/IEC 27001 establish authoritative requirements for cryptographic protection, lifecycle management, cybersecurity controls, and organizational governance [38–42]. These documents provide indispensable baselines and regulatory legitimacy. However, as discussed in Sect. 2.2, they are developed largely within independent standards bodies and application domains. This often leads to overlapping requirements, inconsistent abstraction levels, and terminology differences across domains. In practice, practitioners must reconcile these differences manually when evaluating complex systems that span multiple standards, which may introduce redundancy and interpretation inconsistencies.

The proposed framework does not replace standards but embeds them within a layered architecture that consolidates heterogeneous requirements into unified, traceable elements. By preserving cross-standard lineage while eliminating duplication, it transforms standards from parallel compliance artifacts into structured inputs within a coherent assurance model.

8.3.2 Academic assurance frameworks and taxonomies

Academic research has produced numerous taxonomies, domain-specific models, and surveys addressing hardware security mechanisms, subsystem heterogeneity, lifecycle trust management, verification methodologies, and quantitative security metrics. As summarized in Sect. 2.3 and Table 1, these contributions significantly advance individual assurance dimensions. However, many studies focus on isolated aspects of assurance rather than providing an integrated evaluation architecture.

Surveys by Yaacoub et al. [45] and Shukla et al. [5] explicitly observe that existing approaches tend to fall into two categories: highly abstract conceptual models that provide limited operational guidance, or narrowly scoped technical solutions targeting specific hardware domains. Consequently, these approaches offer valuable insights but limited support for integrating heterogeneous assurance artifacts across system boundaries.

Furthermore, prior quantitative and AI-driven proposals for security scoring often operate at a flat abstraction level. They typically focus on vulnerability metrics, resilience scoring, or automated requirement extraction without embedding these metrics within a unified hierarchical assurance structure. Requirement definition, verification reasoning, and scoring mechanisms are therefore frequently treated as separate analytical layers.

The framework proposed in this study addresses this gap by integrating subsystem decomposition, standards-derived requirements, verification linkage, and aggregation semantics within a single architecture. Quantitative indicators are not introduced as standalone metrics but emerge from structured requirement-to-verification mappings, thereby preserving traceability across multiple levels of abstraction.

8.3.3 Hardware security evaluation models

Hardware-focused evaluation schemes, including NIST SP 800-193 guidance on firmware resilience [29], TCG device trust models [62], and certification-oriented models such as Common Criteria [23], provide rigorous technical guidance

and structured evaluation processes. These approaches are essential for validating specific security properties such as hardware roots of trust, secure boot mechanisms, and recovery capabilities.

However, these models are typically certification-oriented and scoped to predefined protection profiles or device classes. While they provide deep technical validation within these boundaries, they offer limited mechanisms for consolidating heterogeneous standards, comparing subsystem assurance results, or aggregating verification outcomes across complex multi-component architectures.

The proposed framework complements these models by introducing explicit subsystem decomposition, cross-standard integration, and quantification-ready aggregation semantics. Rather than focusing solely on certification artifacts, the framework enables structured comparison across evaluation units and supports analytical reasoning at the system-level while preserving linkage to underlying verification evidence.

8.3.4 Synthesis

Across these streams of work, a recurring limitation is the separation between requirement specification, verification demonstration, and quantitative reasoning. Standards define obligations, assurance methodologies structure evaluation arguments, certification schemes validate technical properties, and scoring frameworks provide numerical indicators. However, these elements are rarely integrated within a single traceable architecture.

The proposed framework integrates these dimensions by:

- consolidating heterogeneous standards within a boundary-driven structure,
- preserving objective-to-verification traceability,
- introducing aggregation semantics that support quantification-ready subsystem roll-up, and
- maintaining explicit epistemic boundaries between structural normalization and empirical calibration.

Table 14 summarizes the principal distinctions between the proposed framework and existing approaches.

8.4 Limitations

While the evaluation demonstrates structural coherence and operational feasibility, several limitations constrain the present study.

First, the empirical basis is limited to a single case study involving a wireless fingertip pulse oximeter. Although the device presents heterogeneous assurance challenges spanning cryptographic protection, subsystem integration, lifecycle processes, and wireless communication, a single instantiation cannot establish statistical generalizability. Broader validation across domains such as industrial IoT, automotive systems, aerospace platforms, or defense hardware is required to assess transferability under differing architectural, regulatory, and threat conditions.

Second, the quantitative extension introduced in Sect. 7 remains methodological rather than empirically calibrated. The aggregation structure, weighting semantics, and illustrative scores demonstrate feasibility and traceability, but they do not yet constitute a validated assurance metric. Weight assignments inherently involve structured judgment, whether standards-based, risk-informed, expert-driven, or uniform. Although sensitivity analysis indicates robustness under bounded perturbations, alternative weighting rationales may influence prioritization outcomes. Empirical calibration using structured elicitation, regulatory benchmarks, operational incident datasets, or cross-system benchmarking remains pending.

Third, industrial-scale deployment has not yet been conducted. The framework has not been evaluated under large, multi-organizational assurance workflows where coordination overhead, documentation burden, and tooling integration may introduce practical constraints. Real-world adoption may reveal scalability challenges not observable within a single-case evaluation.

Finally, the present scope emphasizes device-level hardware-software integration within a clearly defined technical boundary. While supply-chain dependencies and governance dimensions are analytically modeled, comprehensive socio-technical evaluation of manufacturing processes, organizational assurance maturity, and lifecycle governance remains outside the current implementation. These dimensions materially influence system trustworthiness and warrant systematic integration in future iterations.

8.5 Future work

Several research directions follow naturally from the present limitations.

First, multi-domain validation is required to evaluate transferability and robustness across heterogeneous regulatory and operational contexts. Application to domains such as industrial control systems, automotive electronics, aerospace hardware, or defense platforms would test the framework's adaptability to differing assurance cultures, certification regimes, and architectural complexities. Comparative cross-domain studies would also enable refinement of abstraction levels and weighting strategies.

Table 14 Comparison of assurance capabilities across existing approaches and the proposed framework

Approach	Core Assurance Capability	Structural Limitations	Advancement in the Proposed Framework
Standards-based assurance (e.g., FIPS 140–3, IEC 62304, ISO 14971)	Defines authoritative security and lifecycle requirements	Standards developed independently across domains; overlapping requirements; inconsistent abstraction levels; manual cross-standard reconciliation	Consolidates heterogeneous standards within a layered architecture and preserves explicit cross-standard traceability across assurance levels
Academic taxonomies and survey frameworks	Classifies hardware security mechanisms and assurance dimensions	Often conceptual or domain-specific; limited operational guidance; weak linkage to verification artifacts	Introduces a domain-agnostic structure that connects subsystem decomposition, requirement derivation, and verification evidence within a single architecture
Verification-focused methods (e.g., adversarial testing, fault injection)	Provides empirical validation of specific security properties	Testing artifacts frequently disconnected from system-level assurance objectives; limited traceable linkage to requirements	Establishes an objective → requirement → verification chain that ensures system-level goals propagate to testable evidence
Quantitative scoring systems (e.g., vulnerability metrics, maturity models)	Produces numerical indicators of security posture	Typically flat abstraction; detached from requirement lineage; limited integration with verification evidence	Embeds quantification-ready aggregation within a hierarchical requirement-to-verification structure that preserves contextual interpretation
Hardware evaluation models (e.g., Common Criteria, NIST SP 800–193, TCG trust models)	Provides structured certification and evaluation processes for hardware security	Certification-bound scope; limited subsystem comparability; weak mechanisms for cross-standard consolidation or system-level aggregation	Enables subsystem decomposition and structured comparison across evaluation units while supporting hierarchical roll-up of verification evidence

Second, empirical calibration of the quantitative model represents a critical next step. Structured expert elicitation, regulatory co-assessments, cross-device benchmarking, and correlation studies using vulnerability datasets such as CWE or CVE may provide data for parameter tuning. Longitudinal operational incident data could further inform validation of aggregation semantics. Such staged calibration would transform the framework from a quantification-ready structure into an empirically grounded assurance methodology.

Third, industrial pilot deployments are necessary to assess scalability and integration. Embedding the framework within existing assurance workflows, toolchains, and certification processes would reveal adoption barriers, coordination challenges, and documentation overhead. Lessons from such pilots would inform refinement of automation interfaces and governance alignment.

Fourth, automation and AI-assisted techniques present opportunities to enhance scalability and reproducibility. Natural language processing could support requirement consolidation across heterogeneous standards, automated traceability validation, and gap detection. However, such integration must preserve boundary clarity and prevent opaque inference from undermining assurance transparency.

Finally, extending the framework toward continuous and adaptive assurance represents a promising direction. Modern systems evolve through frequent updates, dynamic configurations, and evolving regulatory requirements, making episodic certification insufficient for long-term assurance. Future work could explore integration with lifecycle management systems, digital twin environments, and continuous compliance monitoring mechanisms that track evolving standards and automatically flag impacted evaluation units. Such extensions would enable ongoing verification and adaptive assurance while maintaining clearly defined evaluation boundaries.

9 Conclusion

This paper introduced a structured and traceable framework for hardware security assurance designed to address fragmentation and ambiguity in current evaluation practices. Grounded in the design principles of progression, scoping, traceability, and analytic reasoning, the framework organizes assurance into six hierarchical layers that progressively refine system boundaries, security objectives, requirements, and verification conditions. By linking high-level protection goals to observable verification evidence within a single structured architecture, the framework establishes a transparent reasoning pathway that improves the clarity and reproducibility of assurance evaluations.

The framework was instantiated through a case study involving a wireless fingertip oximeter, a representative safety–critical device operating within healthcare infrastructure. The case study demonstrated how obligations drawn from heterogeneous standards, including IEC, UL, and FIPS guidance, can be consolidated into a coherent and traceable requirement structure. Verification conditions derived from these requirements illustrated how abstract assurance criteria can be connected to observable technical evidence, such as firmware integrity validation and tamper-event logging. Comparative analysis against a standards-only baseline further highlighted improvements in structural coherence, coverage transparency, and diagnostic interpretability.

The quantitative methodology presented in Sect. 7 extends the framework by introducing a structured mechanism for aggregating subsystem-level verification outcomes. While the scoring model remains illustrative and requires empirical calibration, the results demonstrate how traceable verification artifacts can support quantification-ready aggregation and structured reasoning about system-level assurance posture. This capability provides a foundation for future analytical evaluation of complex infrastructures composed of heterogeneous hardware subsystems.

Taken together, the proposed framework contributes a systematic approach for integrating architectural decomposition, standards-derived requirements, verification evidence, and analytical aggregation within a single assurance structure. By clarifying evaluation boundaries and preserving traceability across assurance artifacts, it offers a practical pathway toward more transparent and structured hardware security assurance. Future work will focus on empirical calibration of the quantitative model, validation across additional safety–critical domains, and exploration of continuous assurance mechanisms that support evolving systems and regulatory environments.

Appendix A: Consolidated assurance requirements derived from selected standards

Evaluation Unit	Security Objective (L3)	Requirement (L5)	Referenced Standard(s)
U_{SENSE}(sensing chain)	Ensure integrity and reliability of physiological measurement	R1. The sensor must validate calibration data before use	IEC 60601–1, FDA guidance
		R2. The signal chain must apply noise filtering with documented tolerance thresholds	IEC 60601–1
		R3. The sensing module must protect against spoofing inputs (e.g., disconnected probes, simulated signals)	UL 2900–2-1
		R4. Sensor firmware must support secure updates with integrity verification	IEC 62304
		R5. Error conditions (out-of-range values, failed calibration) must be logged and signaled to the CPU	IEC 60601–1
U_{CPU}(compute & firmware)	Maintain trusted boot and runtime integrity	R6. The CPU must verify firmware signatures before execution	FIPS 140–3
		R7. Rollback to previous firmware versions must be prevented unless explicitly authorized	UL 2900–2-1
		R8. The CPU must enforce memory protection to isolate code and data regions	IEC 62304
		R9. Input data from peripherals must undergo validation before processing	UL 2900–2-1
		R10. The CPU must implement secure error handling to avoid exposing sensitive state information	IEC 62304
		R11. Audit logs must be generated for boot integrity checks and runtime faults	IEC 62304, FDA guidance

Evaluation Unit	Security Objective (L3)	Requirement (L5)	Referenced Standard(s)
U_{BLE}(wireless interface)	Protect confidentiality and authenticity of communication	R12. BLE traffic must be encrypted using FIPS-approved algorithms	FIPS 140–3
		R13. Device pairing must require mutual authentication	UL 2900–2-1
		R14. Session keys must be generated using approved random number generators	FIPS 140–3
		R15. Failed authentication attempts must trigger exponential back-off or lockout	UL 2900–2-1
		R16. The BLE stack must detect and handle denial-of-service attempts (e.g., repeated connection floods)	UL 2900–2-1
U_{PHYS}(physical & power integrity)	Ensure resilience against physical tampering and fault injection	R17. The device enclosure must resist undetected opening attempts	IEC 60601–1
		R18. Tamper events must trigger secure erasure of cryptographic material	FIPS 140–3
		R19. The system must maintain safe state in case of power fluctuations or brown-outs	IEC 60601–1
		R20. Power supply integrity must be monitored and logged continuously	IEC 60601–1
		R21. Fault injection attempts (e.g., clock glitches) must be mitigated with redundancy or detection	UL 2900–2-1
		R22. Debug/test interfaces must be disabled or protected in deployed devices	UL 2900–2-1, IEC 62304

IEC 62304, UL 2900–2-1, FIPS 140–3, IEC 60601–1, FDA guidance

Appendix B: Worked example of supply chain analytics

Evaluation Unit	Responsible Party	Raw Contribution (%)	Criticality Weight (λq)	Raw $\times \lambda q$	Weighted Contribution (%)
U_{CPU}	Vendor	52.5	1.0	52.5	52.5
U_{PHYS}	Vendor	47.5	1.0	47.5	47.5

Evaluation Unit	Responsible Party	Raw Contribution (%)	Criticality Weight (λq)	Raw × λq	Weighted Contribution (%)
U_{SENSE}	Supplier	52.0	1.35	70.2	59.4
U_{BLE}	Supplier	48.0	1.0	48.0	40.6
Total	–	200.0	–	218.2	200.0

Explanation

- Raw Contribution (%): Share before weighting. Vendor sums to 100%, Supplier sums to 100% separately → 200 total across both categories.
- Raw × λq: Intermediate step showing the effect of applying weights.
- Weighted Contribution (%): Normalized so Vendor = 100% and Supplier = 100% within category, i.e.,
- U_{SENSE} dominates Supplier after weighting.

Acknowledgements The authors gratefully acknowledge the support and collaboration of partners in the SFI NORCICS consortium.

Author contributions S.-F.W.—Conceptualization, methodology design, software, data collection, AI-assisted clustering implementation, threat validation, formal analysis, figure generation and manuscript drafting. A.S.—Expert review of taxonomy consolidation, methodological validation, interpretation of results and critical manuscript revision. All authors have read and agreed to the published version of the manuscript.

Funding Open access funding provided by NTNU Norwegian University of Science and Technology (incl St. Olavs Hospital - Trondheim University Hospital). This work has received funding from the Research Council of Norway through the SFI Norwegian Centre for Cybersecurity in Critical Sectors (NORCICS) project no. 310105. Open access funding provided by NTNU Norwegian University of Science and Technology (incl St. Olavs Hospital—Trondheim University Hospital).

Data availability All data generated or analyzed during this study are included in this article. Any additional information (e.g., intermediate analysis scripts or aggregation notes) is available from the corresponding author upon reasonable request.

Declarations

Conflict of interest The authors declare no conflicts of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Williams, P.A., Woodward, A.J.: Cybersecurity vulnerabilities in medical devices: a complex environment and multifaceted problem. Med. Dev. Evidence Res. 305–316 (2015)
2. Anton, S.D.D., et al.: The global state of security in industrial control systems: an empirical analysis of vulnerabilities around the world. IEEE Internet Things J. **8**(24), 17525–17540 (2021)
3. Ul Haq, S., et al.: A survey on IoT & embedded device firmware security: architecture, extraction techniques, and vulnerability analysis frameworks. Discover Internet of Things **3**(1), 17 (2023)
4. Bloomfield, R., Rushby, J.: Assurance 2.0. (2020)
5. Shukla, A., et al.: System security assurance: a systematic literature review. Comput. Sci. Rev. **45**, 100496 (2022)
6. Mohamad, M., Steghöfer, J.P., Scandariato, R.: Security assurance cases—state of the art of an emerging approach. Empir. Softw. Eng. **26**(4), 70 (2021)
7. Macher, G., et al.: Integrated safety and security development in the automotive domain. SAE Technical Paper (2017)
8. Djebbar, F., Nordström, K.: A comparative analysis of industrial cybersecurity standards. IEEE Access **11**, 85315–85332 (2023)
9. Wei, R., et al.: ACCESS: assurance case centric engineering of safety–critical systems. J. Syst. Softw. **213**, 112034 (2024)
10. Halak, B.: Cist: A threat modelling approach for hardware supply chain security. In: Hardware supply chain security: threat modelling, emerging attacks and countermeasures, pp. 3–65. Springer (2021)
11. Akter, S., Khalil, K., Bayoumi, M.: A survey on hardware security: current trends and challenges. IEEe Access **11**, 77543–77565 (2023)
12. Wen, S.F., Shukla, A., Katt, B.: Developing security assurance metrics to support quantitative security assurance evaluation. J. Cybersecur. Privacy **2**(3), 587–605 (2022)
13. Budde, C.E., Stoelinga, M.: Efficient algorithms for quantitative attack tree analysis. In: 2021 IEEE 34th Computer Security Foundations Symposium (CSF). IEEE (2021)
14. Idika, T., Akturk, I.: Holistic hardware security assessment framework: a microarchitectural perspective. arXiv preprint arXiv:2110.09849, (2021)
15. Reimann, L.M., et al.: Qflow: quantitative information flow for security-aware hardware design in verilog. In: 2021 IEEE 39th International Conference on Computer Design (ICCD). IEEE (2021)
16. Bjarnason, E., Wnuk, K., Regnell, B.: Requirements are slipping through the gaps—a case study on causes & effects of communication gaps in large-scale software development. In: 2011 IEEE 19th International Requirements Engineering Conference. IEEE (2011)
17. Basili, V.R., Caldiera, G., Rombach, H.D.: The Goal Question Metric Paradigm. University of Maryland, Department of Computer Science (1994)
18. Goodenough, J.B., Weinstock, C.B., Klein, A.Z.: Toward a theory of assurance case confidence. (2012)
19. Nielsen, C.B., et al.: Systems of systems engineering: basic concepts, model-based techniques, and research directions. ACM Comput. Surveys (CSUR) **48**(2), 1–41 (2015)
20. Spears, J.L., Barki, H., Barton, R.R.: Theorizing the concept and role of assurance in information systems security. Inf. Manag. **50**(7), 598–605 (2013)
21. Mansourov, N., Campara, D.: System assurance: beyond detecting vulnerabilities. Elsevier (2010)
22. Bagam, N., et al.: Advancements in quality assurance and testing in data analytics. J. Comput. Anal. Appl. **33**(8) (2024).

23. ISO, ISO/IEC 15408-1:2022 information security, cybersecurity and privacy protection—evaluation criteria for IT security part 1: introduction and general model. Version 3.1, Release 5 ed. Geneva: International Organization for Standardization (ISO/IEC) (2022)
24. NIST, NIST SP 800-160 developing cyber-resilient systems: a systems security engineering approach. (2021). Available from: https://csrc.nist.gov/pubs/sp/800/160/v2/r1/final
25. Wen, S.-F., Katt, B.: A quantitative security evaluation and analysis model for web applications based on OWASP application security verification standard. Comput. Secur. **135**, 103532 (2023)
26. Katt, B., Prasher, N.: Quantitative security assurance. In: Exploring security in software architecture and design. IGI Global. p. 15–46 (2019)
27. Sharma, A., Wen, S.F.: A quantitative framework for assessing and enhancing hardware security resilience. In: 2024 IEEE Physical Assurance and Inspection of Electronics (PAINE). IEEE (2024)
28. Mishra, J., Sahay, S.K.: Modern hardware security: a review of attacks and countermeasures. arXiv preprint arXiv:2501.04394, (2025)
29. NIST, NIST IR 8517 hardware security failure scenarios: potential weaknesses in hardware design. National Institute of Standards and Technology (NIST) (2024)
30. Shepherd, C., et al.: Physical fault injection and side-channel attacks on mobile devices: a comprehensive analysis. Comput. Secur. **111**, 102471 (2021)
31. Socha, P., Miškovský, V., Novotný, M.: A comprehensive survey on the non-invasive passive side-channel analysis. Sensors **22**(21), 8096 (2022)
32. Knechtel, J., et al.: Towards secure composition of integrated circuits and electronic systems: on the role of EDA. arXiv preprint. (2020). arXiv:2001.09672
33. Nabeel, M., et al.: 2.5 D root of trust: Secure system-level integration of untrusted chiplets. IEEE Trans. Comput. **69**(11): 1611–1625 (2020)
34. Höfig, K., Zeller, M., Heilmann, R.: ALFRED: a methodology to enable component fault trees for layered architectures. In: 2015 41st Euromicro Conference on Software Engineering and Advanced Applications. IEEE (2015)
35. Collier, Z.A., Lambert, J.H.: Managing obsolescence of embedded hardware and software in secure and trusted systems. Front. Eng. Manag. **7**(2), 172–181 (2020)
36. (IMDRF), I.M.D.R.F.: Principles and Practices for the Cybersecurity of Legacy Medical Devices. (2023)
37. Zhang, D., et al.: A hardware design language for timing-sensitive information-flow security. Acm Sigplan Notices **50**(4), 503–516 (2015)
38. NIST, NIST FIPS 140–3: Security Requirements for Cryptographic Modules. (2019). Available from: https://csrc.nist.gov/pubs/fips/140-3/final
39. IEC, IEC 62304:2006—Medical device software—Software life cycle processes. (2006), IEC: Geneva
40. ISO, ISO 14971:2019 Medical devices—application of risk management to medical devices. (2019). Available from: https://csrc.nist.gov/pubs/fips/140-3/final
41. Laboratories, U.: UL 2900-2-1:2017. Standard for software cybersecurity for network-connectable products—Part 2–1: Particular requirements for network connectable components of healthcare systems. (2017). Available from: https://csrc.nist.gov/pubs/fips/140-3/final
42. ISO, ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection—information security management systems—Requirements. 2022. Available from: https://csrc.nist.gov/pubs/fips/140-3/final
43. Kocher, P., et al.: Spectre attacks: exploiting speculative execution. Commun. ACM **63**(7), 93–101 (2020)
44. MITRE. CWE-1247: improper protection against voltage and clock glitches. Common Weakness Enumeration. (2023). Available from: https://cwe.mitre.org/data/definitions/1247.html
45. Yaacoub, J.P.A., et al.: Cyber-physical systems security: limitations, issues and future trends. Microprocess. Microsyst. **77**, 103201 (2020)
46. Frazelle, J.: Securing the boot process: the hardware root of trust. Queue **17**(6), 5–21 (2019)
47. Vidaković, M., Vinko, D.: Hardware-based methods for electronic device protection against invasive and non-invasive attacks. Electronics **12**(21), 4507 (2023)
48. Parisi, E., et al. Assessing the performance of opentitan as cryptographic accelerator in secure open-hardware system-on-chips. In: Proceedings of the 21st ACM International Conference on Computing Frontiers. (2024)
49. Lou, X., et al.: A survey of microarchitectural side-channel vulnerabilities, attacks, and defenses in cryptography. ACM Comput. Surveys (CSUR) **54**(6), 1–37 (2021)
50. Suhail, S., et al.: A framework for enhancing cyber incident response with security-enhancing digital twins in cyber-physical systems. Internet of Things **31**, 101547 (2025)
51. Kazemi, Z., et al.: Hardware security vulnerability assessment to identify the potential risks in a critical embedded application. In: 2020 IEEE 26th International Symposium on On-Line Testing and Robust System Design (IOLTS). IEEE (2020)
52. Bakirtzis, G.: Compositional cyber-physical systems theory. arXiv preprint arXiv:2109.04858, (2021)
53. Kawakami, H., et al.: Hardware security evaluation using assurance case models. In: 2015 10th International Conference on Availability, Reliability and Security. IEEE (2015)
54. Saygın, N., Onay, Z.: An obsolescence management framework for a defense industry company. Procedia CIRP **128**, 686–691 (2024)
55. Rajendran, J., et al.: Nano meets security: exploring nanoelectronic devices for security applications. Proc. IEEE **103**(5), 829–849 (2015)
56. Rostami, M., Koushanfar, F., Karri, R.: A primer on hardware security: Models, methods, and metrics. Proc. IEEE **102**(8), 1283–1295 (2014)
57. Rostami, M., et al.: Hardware security: threat models and metrics. In: 2013 IEEE/ACM International Conference on Computer-Aided Design (ICCAD). IEEE (2013)
58. Kastner, R., Hu, W., Althoff, A.: Quantifying hardware security using joint information flow analysis. In: 2016 design, automation & test in Europe conference & exhibition (DATE). IEEE (2016)
59. Chen, J.L., et al.: Security document generation for common criteria using machine learning and rule-based expert system. In: 2023 IEEE International Conference on Industry 4.0, Artificial Intelligence, and Communications Technology (IAICT). IEEE (2023)
60. Mohamad, M. et al.: Identifying security-related requirements in regulatory documents based on cross-project classification. In: Proceedings of the 18th International Conference on Predictive Models and Data Analytics in Software Engineering. (2022)
61. Wen, S.F., Katt, B.: Exploring the role of assurance context in system security assurance evaluation: a conceptual model. Inf. Comput. Secur. **32**(2), 159–178 (2024)
62. Group, T.C.: Implementing Hardware Roots of Trust, TCG White Paper, (2019)
63. Venable, J., Pries-Heje, J., Baskerville, R.: A comprehensive framework for evaluation in design science research. In: International conference on design science research in information systems. Springer (2012)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.